

REACT PREMIUM NOTES

COMPLETE TOPIC INDEX

NO.	TOPIC	ORIGINAL PAGE	IMPORTANT SUBTOPICS
1	Basics of React JS	1	React overview; JSX; Components; Props; State; Data Flow; Hooks; Virtual DOM; basic example
2	Imports & Exports in React	2–4	Component import/export; CSS files; images; utility modules; JSON data; NPM packages; importing hooks
3	JSX (JavaScript XML)	4–12	JSX expressions; variables/function calls; template literals; inline styles; conditional rendering; loops with map(); JSX as expression; React.createElement(); JSX interview concepts; transpilation vs compilation; className/htmlFor
4	React Components	12–15	Reusable UI components; functional vs class components; props; state; component composition; lifecycle overview; advanced component patterns
5	CSS in React	16–18	Regular/global CSS; CSS Modules; inline CSS; Tailwind CSS setup with Vite
6	React Icons	19	Installation; importing icons; multiple icons; icon usage example
7	Props in React	19–26	Passing props; destructuring; immutability; string/number/boolean/object/array/function props; JSX as props; children prop; default props; spread operator for props
8	Data Flow in React	27–28	One-way data flow; parent-to-child flow; passing callbacks; child-to-parent communication; forceful flow concept
9	State in React JS	29–34	useState(); state syntax/destructuring; primitive/complex state; direct vs functional updates; re-rendering; state snapshots; batching; functional update solution

Page numbers above refer to the ORIGINAL React notes pages. The index is inserted before original page 1.

REACT PREMIUM NOTES — INDEX

NO.	TOPIC	ORIGINAL PAGE	IMPORTANT SUBTOPICS
10	Hooks in React	35–37	Rules of Hooks; built-in hooks overview; useState(); useEffect(); side effects; dependency array; cleanup
11	Event Handling in React	37–42	React vs HTML events; event handlers; SyntheticEvent; event arguments; passing arguments; state snapshot in handlers; propagation; stopPropagation(); event handling patterns
12	Conditional Rendering	43–48	if/else; ternary operator; logical &&; variables; conditional CSS/className; state-based rendering; returning null; advanced component selection; switch; object lookup
13	List Rendering in React	49–52	Array rendering; arrays of objects; arrays of arrays; reusable components with map(); conditional list rendering; keys
14	Forms in React	53–68	Controlled/uncontrolled inputs; multiple fields; checkboxes; grouped checkboxes; select all/deselect all; form submission; radio buttons; grouped radios; select dropdowns; dynamic and dependent dropdowns
15	useRef() Hook	69–70	Refs; DOM access; focus input; mutable values without re-render; persistent .current value
16	Fetch & Axios in React	71–89	Client-server architecture; JSON; HTTP methods; fetch(); GET/POST/PUT/DELETE; Axios installation/configuration; CRUD with Axios; shorthand methods; interceptors; upload progress; Axios instances
17	Routing with React Router DOM	90–102	SPA routing; installation; BrowserRouter; Routes/Route; Link; NavLink; useNavigate(); dynamic routes; useParams(); URL params; Navigate redirects; nested routes; Outlet; index routes; createBrowserRouter; RouterProvider; loaders; useLoaderData()
18	State Management	102–103	Why state management matters; data consistency; cleaner code; debugging; performance; scalability; shared state concepts

REACT PREMIUM NOTES — INDEX

NO.	TOPIC	ORIGINAL PAGE	IMPORTANT SUBTOPICS
19	Context API in React	103–105	Prop drilling; createContext(); Provider; useContext(); custom context provider; custom context hook; production approach
20	Zustand State Management	106–110	Global store; installation; create(); state/actions; using store hooks without Provider; persist middleware; localStorage/sessionStorage; DevTools
21	useReducer() Hook	111–113	When useReducer is useful; reducer; state; dispatch; actions; payload; example implementation
22	Redux with React	114–124	Redux installation; React-Redux; Provider; store; reducers; actions; useSelector(); useDispatch(); multiple reducers; combineReducers(); DevTools; middleware; persistence; Redux Thunk
23	Redux Toolkit (RTK)	125–130	configureStore(); createSlice(); Provider; Immer; slice reducers/actions; async logic; createAsyncThunk(); rejected/fulfilled handling
24	Material UI (MUI)	131	What MUI is; installation; React component library based on Material Design
25	Higher-Order Components (HOC)	132–133	HOC concept; wrapper function; with-style pattern; applying an HOC to a component
26	Custom Hooks	133–134	Reusable hook logic; custom useCounter example; returning state and action functions

Page numbers above refer to the ORIGINAL React notes pages. The index is inserted before original page 1.

REACT PREMIUM NOTES — INDEX

NO.	TOPIC	ORIGINAL PAGE	IMPORTANT SUBTOPICS
27	React Lifecycle	135–138	Mounting; updating; unmounting; class lifecycle methods; useEffect equivalents; cleanup during unmount
28	useLayoutEffect() Hook	138–140	Timing vs useEffect; DOM measurement; tooltip/dropdown positioning; scroll/layout adjustments; avoiding visual flicker
29	React.memo()	141–145	Preventing unnecessary re-renders; shallow prop comparison; custom comparison function; reference stability; stable object references
30	useMemo() Hook	146–147	Memoizing expensive calculations; dependency array; recalculation rules; practical example
31	useCallback() Hook	147–149	Memoizing function references; re-renders; stable callbacks; interaction with memoized child components
32	forwardRef in React	149–150	Passing refs through custom components; parent-to-child DOM ref; legacy React pattern
33	Code Splitting in React	150–154	Component-based splitting; route-based splitting; library splitting; lazy(); Suspense; dynamic import(); loading only required code
34	Virtualization in React	154–156	Large-list rendering; visible-item calculation; scroll handling; slicing visible data; React Virtuoso; React Window

Page numbers above refer to the ORIGINAL React notes pages. The index is inserted before original page 1.

REACT PREMIUM NOTES

Basics Of React JS

- JSX (JavaScript XML)
- Components
- Props (Properties)
- State
- Data Flow
- Hooks
- Virtual DOM

Code Example

```
import React, { useState } from "react";
import imgUrl from "../assets/coding-love.png";
import "../styles/App.css";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div className="p-5">
      <h1>Muhammad Hussain</h1>
      <p>you'll {21 + 1} years old next year</p>

      <ImgComp imageUrl={imgUrl} />

      <h3>Count is {count}</h3>
      <button onClick={() => setCount(count + 1)}>Update Count</button>
    </div>
  );
}
```

```
function ImgComp({ imageUrl }) {
  return (
    <figure>
      <img width={500} src={imageUrl} alt="image is loading..." />
    </figure>
  );
}

export default App;
```

Imports And Export In React JS

Components Import Export

Export component

```
export default function ImgComp({ imageUrl }) {
  return (
    <figure>
      <img width={500} src={imageUrl} alt="image is loading..." />
    </figure>
  );
}
```

Import Component

```
import ImgComp from "../components/ImgComp";
```

using a component

```
<ImgComp imageUrl={imageUrl} />
```

Import CSS File

Import a CSS file

```
import "../styles/App.css";
```

Import Images

Import images

```
import imgUrl from "../assets/coding-love.png";
```

using an imported image

```
<img src={imgUrl} alt="image is loading..." />
```

Modules / Function / Utility

Export modules or functions or utilities

```
export function foramteDate(date) {  
  let formattedDate = date.toLocaleDateString();  
  return formattedDate;  
}  
  
export function displayDate(date) {  
  return `Formatted Date is ${date}`;  
}
```

Import modules or functions or utilities

```
import { foramteDate, displayDate } from "../utilities/date-  
fomat-helper.js";
```

Using modules or functions or utilities

```
<p>{displayDate(foramteDate(date))}</p>
```

Import JSON Data

Import JSON file

```
import users from "../data/userData.json";
```

Third-party Packages (NPM Modules)

Import third party packages

```
import axios from "axios";  
import { BrowserRouter } from "react-router-dom";
```

Import Hooks

Import Hooks

```
import React, { useState, useEffect } from "react";
```

JSX (JavaScript XML)

JSX (JavaScript XML) is a syntax extension for JavaScript used in React. It lets you write HTML-like code inside JavaScript. React look like HTML but behave like JavaScript.



JSX is not HTML — it's syntactic sugar over `React.createElement()`

Embedding Expressions

Valid expression to embed

- Variables
- Function Calls
- Ternary Expressions
- Mathematical Operations

Statements like if-else, for, etc. are **not allowed**.



JSX and template literals Only expressions can go inside `{}` or `${}` and not statements.

```
import React from "react";

function App() {
  let name = "Hussain";

  let isStudent = true;

  function sum(a, b) {
    return a + b;
  }

  return (
    <div className="p-5">
      <h1>Hye! {name}</h1>
      <p>You'll {21 + 1} years old next year</p>
      <p>sum of 4 and 7 is {sum(4, 7)}</p>
      <p>You are {isStudent ? "student" : "not student"}</p>
    >
  </div>
);
}

export default App;
```

JSX With Inline Styles

JSX style is an object, not a string

```
<h1
  style={{
    padding: "10px",
    backgroundColor: "#121212",
    borderRadius: "5px",
```

```
    }}  
>  
    Muhammad Hussain  
</h1>
```

another method for inline style

```
<h1  
  style={{  
    "padding": "10px",  
    "background-color": "#121212",  
    "border-radius": "5px",  
  }}  
>  
  Muhammad Hussain  
</h1>
```

Conditional Rendering In JSX

Using Ternary Operator

```
import React from "react";  
  
function App() {  
  const isLoggedIn = true;  
  
  return (  
    <div className="p-5">  
      <h1>{isLoggedIn ? "Welcome Back!" : "Please login."}  
    </h1>  
    </div>  
  );  
}  
  
export default App;
```

Short-circuit rendering

```
import React from "react";

function App() {
  const isAdmin = true;

  return (
    <div className="p-5">
      <h1>{isAdmin && "Admin Panel"}</h1>
    </div>
  );
}

export default App;
```

JSX With Loops (Arrays)

JSX doesn't support for loop, but we can use `.map()`.

```
import React from "react";

function App() {
  const heros = ["Hussain", "Suleman", "Zaryab"];

  return (
    <div className="p-5">
      <ul className="list-disc list-inside">
        {heros.map((name, index) => (
          <li key={index}>{name}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

JSX Is An Expression

JSX is an expression, you can

- Assign it to variable
- Pass it as a props
- Return it from a function

Assigning to Variable

```
let desc = <p>This is a content description</p>;
```

Passing it as Props

```
<Header  
  data={  
    <div>  
      <p>Hye! I'm Hussain</p>  
      <p>I'm 21 years old</p>  
    </div>  
  }  
</Header>
```

Behind The Scenes

JSX Compiled to `React.createElement()` behind the scenes.

JSX

```
const element = <p className="username">Muhammad Hussain</p>;
```

Compiled into

```
const element = React.createElement("p", { className: "username" }, "Hussain Ilyas");
```

code example

```

import React from "react";

function App() {
  const user = { name: "Hussain", age: 21 };

  return (
    <div className="p-5">
      { /* === JSX === */ }
      <div className="card" style={{ color: "red", fontSiz
e: "20px" }}>
        <h1>Hye! {user.name}</h1>
        <p>You are {user.age} years old</p>
      </div>

      { /* === Behind the scene === */ }
      {React.createElement(
        "div",
        { className: "card", style: { color: "red", fontSiz
e: "20px" } },
        React.createElement("h1", null, `Hye! ${user.name}
`),
        React.createElement("p", null, `You are ${user.age}
years old`),
      )}
    </div>
  );
}

export default App;

```

JSX Rules

- Must return single parent element. Wrap with div `<div></div>` or fragments `<></>`
- Use `className` and `htmlFor` props instead of `class` and `for` attribute. (`class` & `for` are reserved keywords in JS)
- Self-closing tags are used for elements without children.

- Cannot use `if-else` directly — use ternary or logical `&&`
- JSX supports only expressions inside `{ }` – not statements.

React JS Interview Questions

What is JSX?

JSX is a syntax extension for JavaScript used in React. It lets you write HTML-like code inside JavaScript. many websites claim that JSX stands for JavaScript XML but React official documentation refers to JSX as a "syntax extension for JavaScript" or "an XML-like extension to the JavaScript language."

Transpilation v/s Compilation

Transpilation	Compilation
Converting code from one version of a language to another (like JSX to JS, or ES6 to ES5).	Converting human-readable code into optimized, often lower-level code (e.g., bundling, minifying).

What is Babel?

A JavaScript transpiler that transforms modern JS/JSX into a form that browsers can understand (usually older JavaScript).

What Happens with JSX behind the scene?

Babel Transpiles JSX → `React.createElement()`

Babel plugin (`@babel/preset-react`) converts JSX syntax into pure JS. This is transpilation because JSX is just syntactic sugar for function calls, not a new language.

Babel Transpiles ES6+ → **ES5**

If you used modern JS features, Babel also converts that into older JavaScript for compatibility.

Bundler Compilation (`webpack` , `vite` , etc.)

The transpiled files are bundled into a single/minified JS file. This step is closer to compilation since your project is being turned into something executable by the browser.

Browser Execution

`React.createElement()` creates virtual DOM objects. React reconciler diffs them. React DOM updates the real DOM.

Why we use double curly braces in style attribute?

Since JSX inline style is an object and not a string. The first pair indicates you are embedding a JavaScript expression inside JSX. The second pair is the object itself

```
<h1 style={{ color: "red", fontSize: "20px" }}>Hye! Hussain</h1>
```

Why we need to use `className` , `htmlFor` instead of `class` and `for` in JSX ?

Since, JSX is ultimately compiled into JavaScript, and we all know that `class` and `for` are reserved keywords in JavaScript. So It would create naming conflicts and leads to errors when the JSX is compiled into JavaScript. That is why `className` & `htmlFor` is used as an alternative to avoid these conflicts.

`className` is attribute or props in JSX?

`className` looks like an HTML attribute because we write it inside a tag. But under the hood, JSX is just JavaScript. When Babel transpiles this, it becomes

```
React.createElement("p", { className: "username" }, "Hussain Ilyas");
```

So, `className` in JSX is a prop, not a raw attribute. But React uses it to set the actual HTML attribute class on the DOM element.

JSX doesn't support loop like `for`, but we can use `map()`. Why?

`for` is a statement, produces no value → JSX can't directly use it.

`map` is an expression, produces an array of elements → JSX can render it.

React JS Components

A component is a reusable piece of UI. Components are Building blocks of every React app. They can be combined to form complex interfaces.

Types Of Components

- Functional Components (Preferred, modern way)
- Class Components (Older, still important historically)

Class Components

Class components are the older style components in React JS, before Hooks. It uses `render()` method. These components are still seen in legacy codebases.

```
import React, { Component } from "react";

class Header extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment = () => {
    this.setState({
      count: this.state.count + 1,
    });
  };

  render() {
```

```

    return (
      <header>
        <h1>This is Header Section</h1>
        <p>The count value is {this.state.count}</p>
        <button onClick={this.increment}>Update Count</butt
on>
      </header>
    );
  }
}

export default Header;

```

Functional Components

A function that returns JSX. No need for `this`, simpler & cleaner. With hooks, functional components replaced most class components.

```

import React, { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div className="p-5">
      <h1>Count is {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment
    </button>
    </div>
  );
}

export default App;

```

Props (Component Inputs)

- Props = Properties

- Passed from parent to child
- Read-only (cannot be changed inside child)
- Props make components dynamic & reusable.

```
import React from "react";

function WelcomeCard(props) {
  return (
    <div>
      <h1>Welcome! {props.name}</h1>
    </div>
  );
}

export default WelcomeCard;
```

Usage

```
<WelcomeCard name="Hussain" />
```

States (Component Memory)

- State = Data that changes inside component
- Managed with `useState()` in functional components
- Props = external input, State = internal memory.

```
import React, { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div className="p-5">
      <h1>Count is {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increment
    </button>
  );
}
```

```
    </div>
  );
}

export default App;
```

Component Composition

- Nesting components inside others
- Makes UI modular
- Small components combined → complex UIs.

Component Lifecycle (class v/s hook)

- Mounting → Updating → Unmounting
- Functional component uses `useEffect()` hook
- Class component uses `componentDidMount()`, `componentDidUpdate()`, `componentWillUnmount()`

Advanced Component Patterns

- Controlled v/s Uncontrolled components
- Higher-Order components (HOC)
- Render Props
- Compound components
- Custom hooks

Best Practices

- Keep components small & focused
- Use `PascalCase` for component names
- Reuse logic with custom hooks instead of duplicating
- Separate UI (presentational) from business logic (container components)

CSS In React JS

Regular CSS File

If you import a CSS file in React (like `App.css`), those styles are global by default.

`App.css` file code

```
.btn {
  padding: 12px 20px;
  background-color: #121212;
  border-radius: 5px;
  cursor: pointer;
  text-transform: capitalize;
  font-size: 16px;
  font-weight: 500;
}

.btn:active {
  scale: 0.98;
}
```

Implementing regular style in JSX file

```
import React from "react";
import "./styles/App.css";

function App() {
  return (
    <div className="p-5">
      <button className="btn">click me</button>
    </div>
  );
}

export default App;
```

CSS Modules

Lets you scope CSS to a component (no global conflicts).

`card.module.css` file code

```
.heading {  
  color: #ccc;  
  font-size: 28px;  
  font-family: Impact, Haettenschweiler, "Arial Narrow Bold", sans-serif;  
}
```

Implementing module style in JSX file

```
import React from "react";  
import Styles from "../styles/card.module.css";  
  
function App() {  
  return (  
    <div className="p-5">  
      <h1 className={Styles.heading}>Hussain</h1>  
    </div>  
  );  
}  
  
export default App;
```

Inline CSS

You use the style prop, which takes a JavaScript object (not a string).

```
import React from "react";  
  
function App() {  
  return (  
    <div className="p-5">  
      <h1 style={{ fontSize: "48px", color: "orange" }}>Hussain</h1>  
    </div>  
  );  
}
```

```
    </div>
  );
}

export default App;
```

Tailwind CSS In React JS

Installation of Tailwind CSS Using vite

Install `tailwindcss` and `@tailwindcss/vite` via npm.

```
npm install tailwindcss @tailwindcss/vite
```

Add the `@tailwindcss/vite` plugin to your `vite` configuration (`vite.config.ts`).

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

Add an `@import` to your CSS file that imports Tailwind CSS (`index.css`).

```
@import "tailwindcss";
```

Setup is established successfully, now run your project

```
npm run dev
```

React JS Icons

Installation

```
npm install react-icons
```

Import Icons

```
import { FaAppleAlt } from "react-icons/fa";
```

Import multiple icons from one icon library

```
import { FaAppleAlt, FaApplePay } from "react-icons/fa";
```

Usage

```
<FaAppleAlt />
```

Complete code example

```
import React from "react";
import { FaAppleAlt, FaApplePay } from "react-icons/fa";

function App() {
  return (
    <div className="p-5">
      <h1>Hussain</h1>
      <FaAppleAlt />
      <FaApplePay />
    </div>
  );
}

export default App;
```

Props In React JS

Props stands for **properties**. They are **inputs** to a React component. Props allow you to **pass data** from a parent component to a child component. They are **read-only** (immutable inside the child component).

```

import React from "react";

function App() {
  return (
    <div className="p-5">
      <Profile username="Hussain" age={21} />
    </div>
  );
}

function Profile(props) {
  console.log(props);
  return (
    <div>
      <h2>{props.username}</h2>
      <p>{props.age} years old</p>
    </div>
  );
}

export default App;

```

Destructuring Props (Cleaner Way)

```

function Profile({ username, age }) {
  return (
    <div>
      <h2>{username}</h2>
      <p>{age} years old</p>
    </div>
  );
}

```

Props Are Immutable

You cannot modify props inside a child component.

```
function Profile(props) {
  props.username = "Hussainian"; // ❌ Not Allowed
  return (
    <div>
      <h2>{props.username}</h2>
    </div>
  );
}
```

String / Number / Boolean as Props

```
import React from "react";

// === Parent Component ===
function App() {
  return (
    <div className="p-5">
      <Profile username="Hussain" age={21} isStudent={true} />
    </div>
  );
}

// === Child Component ===
function Profile({username, age, isStudent}) {
  return (
    <div>
      <h2>{username} is {age} years old.</h2>
      <p>He is {isStudent ? "Student" : "not Student"}</p>
    </div>
  );
}

export default App;
```

Object as Props

```

import React from "react";

// === Parent Component ===
function App() {
  const student = { name: "Suleman", age: 21 };

  return (
    <div className="p-5">
      <Profile user={student} />
    </div>
  );
}

// === Child Component ===
function Profile({ user }) {
  return (
    <div>
      <h2>{user.name} is {user.age} years old.</h2>
    </div>
  );
}

export default App;

```

Array as Props

```

import React from "react";

// === Parent Component ===
function App() {
  const students = ["Hussain", "Suleman", "Zaryab"];

  return (
    <div className="p-5">
      <Profile data={students} />
    </div>
  );
}

```

```

}

// === Child Component ===
function Profile({ data }) {
  return (
    <div>
      {data.map((std, index) => (
        <p key={index}>{std}</p>
      ))}
    </div>
  );
}

export default App;

```

Function as Props

Used for event handling or passing logic.

```

import React, { useState } from "react";

// === Parent Component ===
export default function App() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div className="p-5">
      <Profile count={count} handler={handleClick} />
    </div>
  );
}

// === Child Component ===
function Profile({ count, handler }) {

```

```

return (
  <div>
    <h2>Count value is {count}</h2>
    <button onClick={handler}>Increment</button>
  </div>
);
}

```

JSX as Props

Passing React elements.

```

import React from "react";

// === Parent Component ===
export default function App() {
  return (
    <div>
      <Card content={<p>This is some content</p>} />
    </div>
  );
}

// === Child Component ===
function Card({ content }) {
  return (
    <div>
      <h1>Child Component</h1>
      {content}
    </div>
  );
}

```

Props as Children

Anything between component tags becomes `props.children`.

```

import React from "react";

// === Parent Component ===
export default function App() {
  return (
    <div>
      <h1>Parent Component</h1>
      <Button>Click Me</Button>
    </div>
  );
}

// === Child Component ===
function Button({ children }) {
  return (
    <>
      <button className="btn">{children}</button>
    </>
  );
}

```

Default Props

You can define default values if a prop is not passed.

```

import React from "react";

// === Parent Component ===
export default function App() {
  return (
    <div>
      <UserData name="Hussain" age={21} />
      <UserData age={17} />
    </div>
  );
}

```

```
// === Child Component ===
function UserData({ name = "NONAME", age }) {
  return (
    <div>
      <h1>{name}</h1>
      <p>{age} years old</p>
    </div>
  );
}
```

Spread Operator For Props

Pass all props at once.

```
import React from "react";

// === Parent Component ===
export default function App() {
  const user = { name: "Suleman", age: 22 };
  return (
    <div>
      <UserData {...user} />
    </div>
  );
}

// === Child Component ===
function UserData({ name, age }) {
  return (
    <div>
      <h1>{name}</h1>
      <p>{age} years old</p>
    </div>
  );
}
```

Data Flow In React JS

React is all about data — how it moves from one component to another defines how your app works. React follows one-way data flow (top → down).

Natural Flow



Parent component → Child component

React is designed with **one-way data binding** - data goes from top (parent) → down (child). This is the default and "natural" direction of data flow.

Example Code

Props are the natural way of passing data.

Considered a parent component is

```
import React from "react";
import ChildComp from "../components/ChildComp";

export default function App() {
  let username = "Hussain";
  return (
    <div>
      <ChildComp data={`Hye! ${username}`} />
    </div>
  );
}
```

Child component

```
import React from "react";

export default function ChildComp({ data }) {
  return (
    <div>
      <h1>{data}</h1>
    </div>
  );
}
```

```
);  
}
```

Forceful Flow

Children can't directly change the parent's data (because props are read-only). To send data back, we need to force a way → usually by **passing down a callback function**. That's why it feels "forceful" or "indirect".

Example Code

Considered parent component is

```
import React, { useState } from "react";  
import ChildComp from "../components/ChildComp";  
  
export default function App() {  
  const [name, setName] = useState("Hussain");  
  
  function handleChangeName(name) {  
    setName(name);  
  }  
  
  return (  
    <div>  
      <h1>{name}</h1>  
      <ChildComp handler={handleChangeName} />  
    </div>  
  );  
}
```

Child component

```
import React from "react";  
  
export default function ChildComp({ handler }) {  
  let data = "This is child component data";  
  
  return (  

```

```

    <div>
      <button className="btn" onClick={() => handler(data)}
    >
      Change Name
    </button>
  </div>
);
}

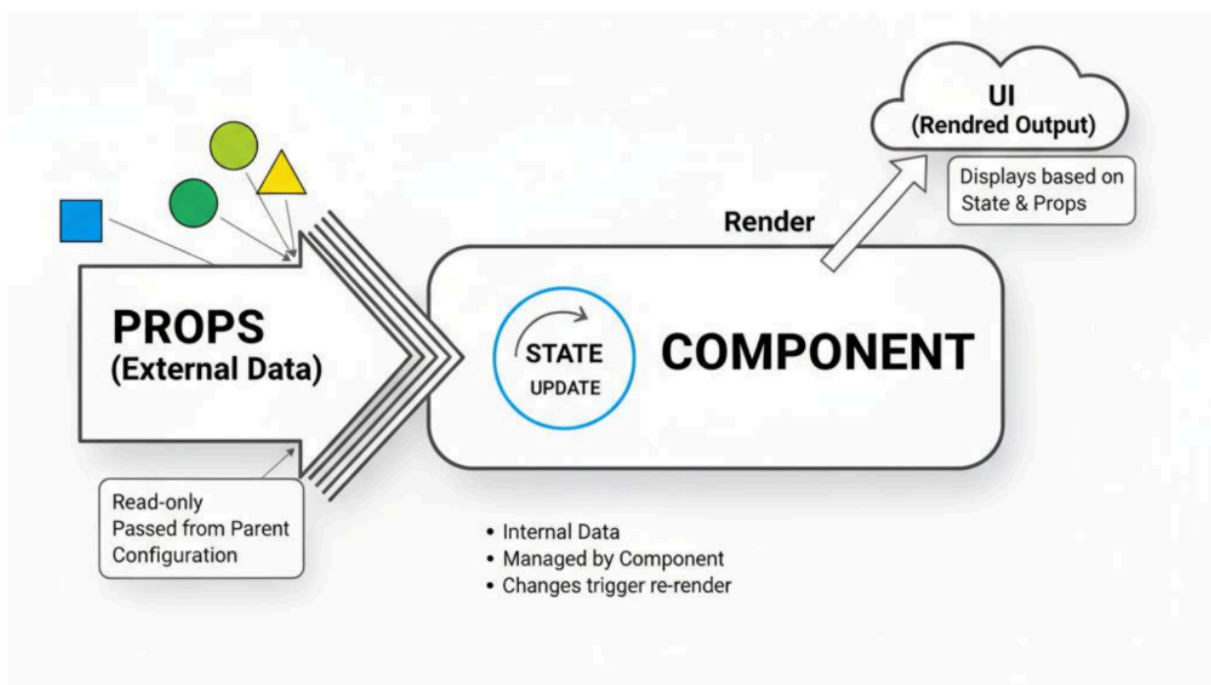
```

State In React JS



State - heart of react app

Change in state directly leads to a change in the UI.



Understanding `useState()` Hook

Import `useState()` hook

```
import React, { useState } from "react";
```

Syntax And Destructuring

```
const [count, setCount] = useState(0);
```

complete full example code

```
import React, { useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Count value is {count}</h1>
      <button className="btn" onClick={() => setCount(count
+ 1)}>
        Increment
      </button>
    </div>
  );
}
```

State Variable

A state variable is a special variable in React that holds data that changes over time and should trigger a re-render of the component when it changes.

State Updater Function

Requests React to update the state and re-render the component. Ensures changes are tracked by internal system of React.



Never modify state directly (`count = count + 1` ❌). Directly mutating the variable won't trigger re-render.

Initial State Possibilities

- Primitive values (number, string, Boolean).
- Complex values (object, array).

Updating State

Direct Update Method

```
setCount(count + 1);
```

Functional Update Method

```
setCount((prev) => {  
  return prev + 1;  
})
```

How React Components Are Re-rendered And Updated

State Declaration

Components in React manage their data using "state." In functional components, this is typically done using the `useState()` hook.

Updating State

When an event occurs (e.g., a user clicks a button, data is fetched), you use the state's updater function (e.g., `setCount()`) to change the state value.

Re-rendering

When the state is updated using its setter function, React detects this change. It then queues a re-render of the component where the state was updated, and any child components that depend on that state.

UI Update

During the re-render, React re-executes the component's render logic (the JSX). It compares the new virtual DOM tree with the previous one and efficiently updates only the necessary parts of the actual DOM to reflect the new state. This ensures that the UI accurately displays the current state values.

Memory (Storage)

Each component instance has its own independent state, even if it's the same component used multiple times.

Why It Works Like This ?

- React doesn't attach state to the component function itself.
- Instead, React keeps a state "**bucket**" for each component instance in the tree.
- When React re-renders, it matches components using their position in the tree and gives them back their own state memory.

Keyed Components

If you render components in a list

```
{items.map((item, index) => (  
  <Counter key={index} />  
))}
```

React uses the key prop to decide which state belongs to which component. If key changes → React treats it as a new component with a fresh state.

async Nature Of State Updates

Updating state doesn't happen immediately - React schedules it and re-renders later.

```
import React, { useState } from "react";  
  
export default function App() {  
  const [count, setCount] = useState(0);
```

```
function handleUpdateCount() {
  setCount(count + 1);
  console.log(count); // will log old value, not updated
}

return (
  <div>
    <h1>Count value is {count}</h1>
    <button className="btn" onClick={handleUpdateCount}>
      Increment
    </button>
  </div>
);
}
```

Batching Of Updates

React batches multiple state updates in the same event to improve performance (avoids unnecessary re-renders).

```
const [count, setCount] = useState(0);

function handleUpdateCount() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}
```

Here, count will increase only by 1, not 3 (because React batches them, and each line uses old count).

How React Schedules It ?

When you call `setState` function

- React adds the update to an internal queue.
- React waits until the current event handler finishes (so it can batch multiple updates).

- Then React decides when to re-render the component.
- During render, React reconciles (diffs) the new virtual DOM with the old one and updates only what changed.

Solution (Use Functional Update)

```
const [count, setCount] = useState(0);

function handleUpdateCount() {
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);
  setCount((prev) => prev + 1);
}
```

Why we can't use `setCount(count++)`

It not works when you declare state with `const` keyword.

```
const [count, setCount] = useState(0);

function handleUpdateCount() {
  setCount(count++); // ✗ Uncaught TypeError: Assignment
  to constant variable.
}
```

If you want to increment your state like this, then you have to use `let` keyword

```
let [count, setCount] = useState(0);

function handleUpdateCount() {
  setCount(count++); // ✓ Know it works
}
```

Hooks In React JS

Hooks are one of the most powerful features in React. They were introduced in React 16.8 to let us use state and lifecycle methods inside functional components — without writing a class.

Before hooks

- Functional components were stateless.
- We used class components for state & lifecycle.

After hooks

- Functional components can now manage state, handle side effects, and reuse logic easily.



Hooks must start with “use” (e.g., `useState()`, `useEffect()`).

Rules of Hooks

- Only call hooks at the top level of your component (not inside loops/conditions).
- Only call hooks from React function components or Custom hooks

Commonly Used Built-In Hooks

- `useState()`
- `useEffect()`
- `useRef()`
- `useReducer()`
- `useMemo()`
- `useCallback()`
- `useLayoutEffect()`
- `useContext()`

Understanding `useEffect()` Hook

If `useState()` is for managing state, then `useEffect()` is for **managing side effects** in React. A side effect means any code that affects something outside React's render cycle.

Example Of Side Effects

- Fetching data from an API
- Setting up event listeners (like resize, scroll)
- Working with timers (setInterval, setTimeout)
- Updating the browser's DOM or title manually

Syntax Of `useEffect()`

Import `useEffect()` Hook

```
import React, { useEffect } from "react";
```

syntax of `useEffect()` Hook

```
useEffect(() => {  
  //side effect code here...  
  return () => {  
    // cleanup code (optional)  
  }  
}, [dependencies]);
```

Example Code

```
import React, { useEffect, useState } from "react";  
  
export default function App() {  
  let [count, setCount] = useState(0);  
  
  function handleUpdateCount() {  
    setCount((prev) => prev + 1);  
  }  
}
```

```

useEffect(() => {
  console.log("This is some side effect task");
}, [count]);

return (
  <div>
    <h1>Count value is {count}</h1>
    <button className="btn" onClick={handleUpdateCount}>
      Increment
    </button>
  </div>
);
}

```

Event Handling In React JS

HTML v/s React Events

Events in HTML

```

<button onclick="handleUpdateCount()">
  Increment
</button>

```

Events in React JS

```

<button onClick={handleUpdateCount}>
  Increment
</button>

```

Key Differences

- React uses `camelCase`
- Pass a function reference, not a string
- React events work across browsers using `SyntheticEvent`

Event Handler Basics

Handle basic events like click, change, and submit.

```
import React from "react";

export default function App() {
  // handler function
  function handleClick() {
    alert("Welcome to my website! 😊");
  }

  return (
    <div>
      <button className="btn" onClick={handleClick}>
        click me
      </button>
    </div>
  );
}
```

Accessing Event Objects

```
import React from "react";

export default function App() {
  // accessing event object
  function handleClick(e) {
    console.log(e.target);
  }

  return (
    <div>
      <button className="btn" onClick={handleClick}>
        click me
      </button>
    </div>
  );
}
```

```
);  
}
```

another method

```
import React from "react";  
  
export default function App() {  
  function handleClick(e) {  
    console.log(e.target);  
  }  
  
  return (  
    <div>  
      <button className="btn" onClick={(e) => handleClick  
(e)}>  
        click me  
      </button>  
    </div>  
  );  
}
```

SyntheticEvent In React JS

Event handling in React follows a similar pattern to standard JavaScript event handling but with some key differences, such as using synthetic events for cross-browser compatibility and providing consistent event handling across different elements and browsers.

When you handle events in React, like clicking a button or typing in an input box, React wraps the native browser events in something called Synthetic Event. This Synthetic Event ensures consistent event handling across different browsers.

But after the event handler runs, React pools the event for performance (so `e.target` may become null in `async` code).

```
function handleClick(e) {  
  setTimeout(() => {
```

```
    console.log(e.target);  
  }, 1000);  
}
```

In React 16 and later versions, this problem is no more.

Passing Arguments To The Event Handlers

```
import React from "react";  
  
export default function App() {  
  function userInfo(name, age) {  
    console.log(`${name} is ${age} years old.`);  
  }  
  
  return (  
    <div>  
      <button className="btn" onClick={() => userInfo("Hussain", 21)}>  
        click me  
      </button>  
    </div>  
  );  
}
```

Using State Hook With Events

```
import React, { useState } from "react";  
  
export default function App() {  
  const [count, setCount] = useState(0);  
  
  function updateCountValue() {  
    setCount((prev) => prev + 1);  
  }  
  
  return (  

```

```

    <div>
      <button className="btn" onClick={updateCountValue}>
        count - {count}
      </button>
    </div>
  );
}

```

React re-renders on `setState()` Event handlers capture the current state snapshot



Pitfall - State updates are asynchronous, so never rely on immediate new values.

Prevent Default

```

import React, { useState } from "react";

export default function App() {
  function handleSubmit(e) {
    e.preventDefault(); // Prevent from reload
    console.log("Form is submitted successfully!");
  }

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit" className="btn">
        submit
      </button>
    </form>
  );
}

```

Stop Propagation

```
import React, { useState } from "react";

export default function App() {
  function handleParentClick() {
    console.log("Parent clicked!");
  }

  function handleChildClick(e) {
    e.stopPropagation();
    console.log("Child clicked!");
  }

  return (
    <div onClick={handleParentClick}>
      <button className="btn" onClick={handleChildClick}>
        click me
      </button>
    </div>
  );
}
```

Conditional Rendering In React JS

React doesn't have if/else statements inside JSX. Instead, we use conditional rendering to decide what to show. **Conditional rendering** means displaying UI elements based on certain conditions or states.

Using if...else Statement

```
import React, { useState } from "react";

export default function App() {
  let isLoggedIn = true;

  if (isLoggedIn) {
    return (
      <div>
```

```

        <h1>Welcome User...</h1>
      </div>
    );
  } else {
    return (
      <div>
        <h1>Please Login</h1>
      </div>
    );
  }
}

```

Using Ternary Operator

```

import React, { useState } from "react";

const WelcomeUserComp = () => <h1>Welcome User...</h1>;
const LoggedInComp = () => <h1>Please Login...</h1>;

export default function App() {
  let isLoggeIn = false;

  return(
    <div>
      {isLoggeIn ? <WelcomeUserComp /> : <LoggedInComp />}
    </div>
  )
}

```

Using Logical && Operator

```

import React, { useState } from "react";

const NotifyMessage = () => <h1>You have new message!</h1>;

export default function App() {
  let hasMessage = true;

```

```

return(
  <div>
    {hasMessage && <NotifyMessage />}
  </div>
)
}

```

Using Variables

```

import React, { useState } from "react";

const ComponentA = () => <h1>This is component A</h1>;
const ComponentB = () => <h1>This is component B</h1>;
const ComponentC = () => <h1>This is component C</h1>;

export default function App() {
  let content;
  let option = "a";

  if (option === "a") {
    content = <ComponentA />;
  } else if (option === "b") {
    content = <ComponentB />;
  } else {
    content = <ComponentC />;
  }

  return (
    <div>
      {content}
    </div>
  );
}

```

Conditional CSS & `className`

Considered you have following CSS styles

```
.btn-active {  
  opacity: 1;  
  pointer-events: all;  
}  
  
.btn-inactive {  
  opacity: 0.5;  
  pointer-events: none;  
}
```

Now in React conditional CSS styling is

```
import React, { useState } from "react";  
  
export default function App() {  
  let isActive = false;  
  
  return (  
    <div>  
      <button className={`btn ${isActive ? "btn-active" :  
"btn-inactive"}}`>  
        click me  
      </button>  
    </div>  
  );  
}
```

Conditional Rendering With State

```
import React, { useState } from "react";  
  
export default function App() {  
  const [showMessage, setShowMessage] = useState(true);  
  
  function handleShowMessage() {
```

```

    setShowMessage(!showMessage);
  }

  return (
    <div>
      <button className="btn" onClick={handleShowMessage}>
        {showMessage ? "hide message" : "show message"}
      </button>
      {showMessage && <p>This is secret message</p>}
    </div>
  );
}

```

Returning Null (Hide Completely)

Returning null means “render nothing”. Common in modal or popup components.

```

import React from "react";

function AlertMessage({ show }) {
  if (!show) {
    return null;
  } else {
    return (
      <div className="alert-message">
        <p>This is an alert!</p>
      </div>
    );
  }
}

export default function App() {
  let isShow = false;

  return (
    <div>
      <AlertMessage show={isShow} />
    </div>
  );
}

```

```
    </div>
  );
}
```

Advanced Conditional Rendering

You can even conditionally select components themselves. Useful for routing-like patterns or dynamic layouts.

```
import React, { useState } from "react";

const HomePage = () => <h1>This is Home Page</h1>;
const AboutPage = () => <h1>This is About Page</h1>;
const PageNotFound = () => <h1>404 Page Not Found</h1>;

export default function App() {
  let Component;
  let routeURL = "/contact";

  if (routeURL === "/" || routeURL === "/home") {
    Component = HomePage;
  } else if (routeURL === "/about") {
    Component = AboutPage;
  } else {
    Component = PageNotFound;
  }

  return <Component />;
}
```

Using **switch** Statement

When you have 3 or more conditions, if/else becomes messy. That's when switch makes your code cleaner and more readable.

```
import React, { useState } from "react";

function StatusMessage({ status }) {
```

```

switch (status) {
  case "loading":
    return <p>Loading...</p>;
  case "success":
    return <p>Data loaded successfully ✓</p>;
  case "error":
    return <p>Something went wrong ✗</p>;
  default:
    return <p>Idle state...</p>;
}
}

export default function App() {
  let status = "success";

  return (
    <div>
      <StatusMessage status={status} />
    </div>
  );
}

```

Using Mapping (Object Lookup)

For even more elegant and scalable conditional rendering—you can use object maps to store your UI patterns.

```

import React, { useState } from "react";

function StatusMessage({ status }) {
  const statusMap = {
    loading: <p>Loading...</p>,
    success: <p>Data is loaded successfully ✓</p>,
    error: <p>Something went wrong ✗</p>,
  };

  return statusMap[status] || <p>Idle state...</p>;
}

```

```
export default function App() {
  let status = "loading";

  return (
    <div>
      <StatusMessage status={status} />
    </div>
  );
}
```

List Rendering In React JS

List Rendering means displaying multiple items (like data from an array) in JSX. we use JavaScript's `.map()` method to loop through data and return JSX elements.

Different Types Of Array Rendering

Normal Array

```
import React from "react";

export default function App() {
  const users = ["Hussain", "Suleman", "Zaryab"];

  return (
    <div>
      {users.map((user, index) => (
        <p key={index}>{user}</p>
      ))}
    </div>
  );
}
```

Array Of Objects

```

import React from "react";

export default function App() {
  const userData = [
    { id: 1, name: "Hussain", age: 21 },
    { id: 2, name: "Suleman", age: 22 },
    { id: 3, name: "Zaryab", age: 20 },
  ];

  return (
    <div>
      {userData.map((user, index) => {
        let { id, name, age } = user;
        return (
          <p key={id}>
            {name} - {age} years old
          </p>
        );
      })}
    </div>
  );
}

```

Array Of Arrays

```

import React from "react";

export default function App() {
  const data = [
    ["HTML", "CSS", "JavaScript", "React"],
    ["Node", "MongoDB", "Express"],
  ];

  return (
    <div>
      {data.map((group, i) => (
        <ul key={i}>

```

```

        {group.map((item, j) => (
          <li key={j}>{item}</li>
        ))}
      </ul>
    ))}
  </div>
);
}

```

Rendering Component With `.map()`

Instead of inline JSX, create a reusable component.

```

import React from "react";

function UserCard({ user }) {
  const { name, age } = user;
  return (
    <h2>
      Name is {name} and Age is {age}
    </h2>
  );
}

export default function App() {
  const users = [
    { id: 1, name: "Hussain", age: 21 },
    { id: 2, name: "Suleman", age: 22 },
    { id: 3, name: "Zaryab", age: 20 },
  ];

  return (
    <div>
      {users.map((user) => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  );
}

```

```
);  
}
```

Rendering List With Condition

You can also conditionally show elements during map

```
import React from "react";  
  
export default function App() {  
  const users = [  
    { id: 1, name: "Hussain", age: 21 },  
    { id: 2, name: "Suleman", age: 22 },  
    { id: 3, name: "Zaryab", age: 20 },  
    { id: 4, name: "Hamid", age: 18 },  
    { id: 5, name: "Ayan", age: 6 },  
  ];  
  
  return (  
    <div>  
      {users.map((user) => {  
        return user.age > 20 ? <p>{user.name}</p> : null;  
      })}  
    </div>  
  );  
}
```

Handle Input Field

In React, unlike plain HTML, we don't directly access the DOM using `document.getElementById()`. Instead, React controls the input's value using state — this makes it a **Controlled Component**.

```
import React, { useState } from "react";  
  
export default function App() {  
  const [inputName, setInputName] = useState("");  
}
```

```

return (
  <div className="p-5">
    <input
      type="text"
      placeholder="Enter your name"
      value={inputName}
      onChange={(e) => setInputName(e.target.value)}
    />
  </div>
);
}

```

- value → connected to state
- onChange → updates state
- Without `onChange`, the input becomes read-only

Controlled Components

A controlled component is a form whose input field value is controlled by React's State.

Here's how it works

- Store input field value in state
- Use change handler with input field
- Value attribute attached with state

```

import React, { useState } from "react";

export default function App() {
  const [inputName, setInputName] = useState("");

  console.log(inputName);

  return (
    <div className="p-5">
      <input

```

```

        type="text"
        placeholder="Enter your name"
        value={inputName}
        onChange={(e) => setInputName(e.target.value)}
      />

      <button onClick={() => setInputName("")} className="b
tn">
        Clear value
      </button>

      <p>{inputName}</p>
    </div>
  );
}

```

Uncontrolled Components

Sometimes we don't bind the input to state — we directly access it via `ref`.

```

import React, { useRef } from "react";

export default function App() {
  const inputNameRef = useRef(null);

  function handleClick() {
    console.log(`You type = ${inputNameRef.current.value}`);
  }

  return (
    <div className="p-5">
      <input ref={inputNameRef} type="text" placeholder="type here..." />
      <button className="btn" onClick={handleClick}>
        Clear value
      </button>
    </div>
  );
}

```

```
);  
}
```

Handle Multiple Input Fields

When handling multiple inputs, use one state object - Dynamic key `[name]:`
`value` - Spread operator `...prev` to keep previous data intact

```
import React, { useState } from "react";  
  
export default function App() {  
  const [formData, setFormData] = useState({  
    username: "",  
    userEmail: "",  
  });  
  
  function handleInputChange(e) {  
    let { name, value } = e.target;  
    setFormData((prev) => {  
      return { ...prev, [name]: value };  
    });  
  }  
  
  return (  
    <div className="p-5">  
      <form className="flex flex-col gap-3 items-start mb-5">  
        <input  
          type="text"  
          name="username"  
          placeholder="type name..."  
          value={formData.username}  
          onChange={handleInputChange}  
        />  
  
        <input  
          type="email"  
          name="userEmail"
```

```

        placeholder="type email..."
        value={formData.userEmail}
        onChange={handleInputChange}
      />

      <button
        onClick={() => setFormData({ username: "", userEmail: "" })}
        className="btn"
      >
        Clear Data
      </button>
    </form>

    <div>
      <p>Name = {formData.username}</p>
      <p>Email = {formData.userEmail}</p>
    </div>
  </div>
);
}

```

Handle Checkbox

In React, a checkbox is controlled when its checked state is managed by React state. We don't use `document.querySelector()` — instead, we use `useState()` and `onChange`.

Single Checkbox

```

import React, { useState } from "react";

export default function App() {
  const [isChecked, setIsChecked] = useState(false);

  return (
    <div className="p-5">

```

```

    <label htmlFor="tac">
      <input
        type="checkbox"
        id="tac"
        checked={isChecked}
        onChange={(e) => setIsChecked(e.target.checked)}
      />{ " "}
      Terms and conditions
    </label>
  </div>
);
}

```

Multiple Independent Checkboxes

Sometimes, you have several separate checkboxes, each with its own state.

```

import React, { useState } from "react";

export default function App() {
  const [isHtml, setIsHtml] = useState(false);
  const [isCss, setIsCss] = useState(false);
  const [isJs, setIsJs] = useState(false);

  return (
    <div className="p-5">
      <div className="flex items-center gap-3">
        <label htmlFor="html">
          <input
            type="checkbox"
            id="html"
            checked={isHtml}
            onChange={(e) => setIsHtml(e.target.checked)}
          />{ " "}
          HTML
        </label>
        <label htmlFor="css">
          <input

```

```

        type="checkbox"
        id="css"
        checked={isCss}
        onChange={(e) => setIsCss(e.target.checked)}
      />{ " " }
      CSS
    </label>
    <label htmlFor="js">
      <input
        type="checkbox"
        id="js"
        checked={isJs}
        onChange={(e) => setIsJs(e.target.checked)}
      />{ " " }
      JavaScript
    </label>
  </div>

  <div>
    <p>{isHtml && "HTML Selected"}</p>
    <p>{isCss && "CSS Selected"}</p>
    <p>{isJs && "JavaScript Selected"}</p>
  </div>
</div>
);
}

```

Grouped Checkboxes (Dynamic Handling)

When you have many checkboxes — instead of multiple states, use one object or array.

```

import React, { useState } from "react";

export default function App() {
  const [languages, setLanguages] = useState({
    html: false,
    css: false,
  });
}

```

```

    js: false,
  });

  function handleLanguageSelect(e) {
    let { name, checked } = e.target;

    setLanguages((prev) => {
      return { ...prev, [name]: checked };
    });
  }

  return (
    <div className="p-5">
      <div className="flex items-center gap-3">
        {Object.keys(languages).map((lang, index) => (
          <label htmlFor={lang} key={index}>
            <input
              type="checkbox"
              id={lang}
              name={lang}
              checked={languages[lang]}
              onChange={handleLanguageSelect}
            />{ " " }
            {lang}
          </label>
        ))}
      </div>

      <div>
        <p>{languages.html && "HTML Selected"}</p>
        <p>{languages.css && "CSS Selected"}</p>
        <p>{languages.js && "JavaScript Selected"}</p>
      </div>
    </div>
  );
}

```

Select All / Deselect All - Feature

```
import React, { useState } from "react";

export default function App() {
  const [languages, setLanguages] = useState({
    html: false,
    css: false,
    js: false,
  });

  function handleLanguageSelect(e) {
    let { name, checked } = e.target;

    setLanguages((prev) => {
      return { ...prev, [name]: checked };
    });
  }

  // Check is all selected or not
  // let isAllSelect = Object.values(languages).every((item) => item === true);
  let isAllSelect = Object.values(languages).every(Boolean);
  console.log(isAllSelect);

  return (
    <div className="p-5">
      <div className="flex items-center gap-3">
        {Object.keys(languages).map((lang, index) => (
          <label htmlFor={lang} key={index}>
            <input
              type="checkbox"
              id={lang}
              name={lang}
              checked={languages[lang]}
              onChange={handleLanguageSelect}
            />{ " " }
          )
        )}
      </div>
    </div>
  );
}
```

```

        {lang}
      </label>
    )))

    { /* Select All Feature */ }
    <label htmlFor="sa">
      <input
        type="checkbox"
        id="sa"
        name="sa"
        checked={isAllSelect}
        onChange={(e) =>
          setLanguages({
            html: e.target.checked,
            css: e.target.checked,
            js: e.target.checked,
          })
        }
      />{ " " }
      Select All
    </label>
  </div>

  <div>
    <p>{languages.html && "HTML Selected"}</p>
    <p>{languages.css && "CSS Selected"}</p>
    <p>{languages.js && "JavaScript Selected"}</p>
  </div>
</div>
);
}

```

Handling Checkbox Inside a Form

```

import React, { useState } from "react";

export default function App() {

```

```

const [formData, setFormData] = useState({
  email: "",
  tc: false,
});

function handleInputChange(e) {
  let { name, value, checked, type } = e.target;
  setFormData((prev) => {
    return { ...prev, [name]: type === "checkbox" ? checked : value };
  });
}

function handleFormSubmit(e) {
  e.preventDefault();
  console.log(formData);
}

return (
  <div className="p-5">
    <form onSubmit={handleFormSubmit}>
      <input
        type="email"
        name="email"
        className="input-field"
        placeholder="Enter your email"
        value={formData.email}
        onChange={handleInputChange}
      />

      <label htmlFor="tc">
        <input
          type="checkbox"
          id="tc"
          name="tc"
          checked={formData.tc}
          onChange={handleInputChange}
        />{ " " }
    </form>
  </div>
)

```

```

        Terms and condition
      </label>

      <button className="btn">Submit Form</button>
    </form>
  </div>
);
}

```

Handle Radio Button

Radio buttons let users select only one option from a group. All radio buttons in the same group share the same **name attribute**. In React, we manage which one is selected using state.

Simple Radio Button

```

import React, { useState } from "react";

export default function App() {
  const [gender, setGender] = useState("");

  return (
    <div className="p-5">
      <label htmlFor="male">
        <input
          type="radio"
          id="male"
          name="gender"
          value="male"
          checked={gender === "male"}
          onChange={(e) => setGender(e.target.value)}
        />
        Male
      </label>
    </div>
  );
}

```

```
);  
}
```

Grouping Radio Button

If you have multiple options, map them dynamically instead of repeating code.

```
import React, { useState } from "react";  
  
export default function App() {  
  const [gender, setGender] = useState("");  
  const genders = ["male", "female", "other"];  
  
  return (  
    <div className="p-5">  
      {genders.map((gen, index) => (  
        <label key={index} htmlFor={gen}>  
          <input  
            id={gen}  
            type="radio"  
            name="gender"  
            value={gen}  
            checked={gender === gen}  
            onChange={(e) => setGender(e.target.value)}  
          />  
          {gen}  
        </label>  
      ))}  
    </div>  
  );  
}
```

Example Code

```
import React, { useState } from "react";  
  
export default function App() {
```

```

const [paymentMethod, setPaymentMethod] = useState("UPI");

const payments = [
  { id: "UPI", info: "Instant transfer via upi apps" },
  { id: "Card", info: "Pay using credit or debit card" },
  { id: "COD", info: "Pay when the other arrives" },
];

return (
  <div className="p-5">
    <div className="flex items-center gap-3">
      {payments.map((item, index) => (
        <label key={index} htmlFor={item.id}>
          <input
            id={item.id}
            type="radio"
            name="payment"
            value={item.id}
            checked={paymentMethod === item.id}
            onChange={(e) => setPaymentMethod(e.target.va
lue)}
          />
          {item.id}
        </label>
      ))}
    </div>

    <div>
      <p>Selected Payment Method = {paymentMethod}</p>
      <p>
        Payment Method Info ={" "}
        {
          payments.find((item) => {
            return item.id === paymentMethod;
          }).info
        }
      </p>
    </div>
  </div>
)

```

```

        </div>
      </div>
    );
  }

```

Handle Select Dropdown

The `<select>` tag creates a dropdown menu, and each `<option>` represents a selectable item. In React, we use state to control which value is currently selected.

Simple Select Dropdown

```

import React, { useState } from "react";

function App() {
  const [language, setLanguage] = useState("");

  console.log(language);

  return (
    <div className="p-5">
      <select value={language} onChange={(e) => setLanguage(
(e.target.value))}>
        <option value="">Select Language</option>
        <option value="html">HTML</option>
        <option value="css">CSS</option>
        <option value="javascript">JavaScript</option>
      </select>
    </div>
  );
}

export default App;

```

Important Points

- Don't miss `value` attribute in `<select>`.
- Don't use `defaultValue` in `<select>`. (uncontrolled component)
- Don't use `selected` attribute on `<option>`. (uncontrolled component)
- Always include a default option like `-- Select --` for better UX.

Dynamic Dropdown (Using `.map()`)

If you have many options, it's best to render them dynamically from an array.

```
import React, { useState } from "react";

function App() {
  const [language, setLanguage] = useState("");

  const languages = ["html", "css", "js", "reactjs", "c++", "python"];

  console.log(language);

  return (
    <div className="p-5">
      <select value={language} onChange={(e) => setLanguage(e.target.value)}>
        <option value="">--- Select Language ---</option>
        {languages.map((lang, index) => (
          <option key={index} value={lang}>
            {lang.toUpperCase()}
          </option>
        ))}
      </select>
    </div>
  );
}

export default App;
```

Dependent Select (Cascading)

second dropdown's options depend on which specific item is selected in the first dropdown — also called a **dependent** or **cascading select**.

```
import React, { useState } from "react";

export default function App() {
  const [country, setCountry] = useState("");
  const [city, setCity] = useState("");

  const data = {
    Pakistan: ["lahore", "karachi", "multan", "peshawar"],
    India: ["delhi", "mumbai", "kolkata", "bengaluru"],
    Japan: ["tokyo", "osaka", "kyoto"],
    Germany: ["berlin", "munich", "hamburg"],
  };

  function handleSelectCountry(e) {
    setCountry(e.target.value);
    setCity("");
  }

  function handleSelectCity(e) {
    setCity(e.target.value);
  }

  return (
    <div className="p-5">
      <select value={country} onChange={handleSelectCountry}>
        <option value="">--- Select Country ---</option>
        {Object.keys(data).map((country, i) => (
          <option key={i} value={country}>
            {country.toUpperCase()}
          </option>
        ))}
      </select>
      {country && (
        <select value={city} onChange={handleSelectCity}>
```

```

    <option value="">--- Select City ---</option>
    {data[country].map((city, j) => (
      <option key={j} value={city}>
        {city.toUpperCase()}
      </option>
    ))}
  </select>
)}
</div>
);
}

```

Understanding `useRef()` Hook

What is `ref`

Before hooks, `ref` was used with class components to directly access DOM elements or React elements.

What is `useRef()`

`useRef()` is a Hook that returns a mutable `ref` object whose `.current` property persists across renders. `useRef()` is like a box you can store anything inside (DOM node, number, object, etc.). Changes to `.current` do not cause re-render. It's not reactive, but persistent.

Two Main Uses Of `useRef()`

- Accessing DOM Elements
- Storing Mutable Values (Without Re-render)

Accessing DOM Elements

When you need to directly manipulate a DOM element (e.g., focus, scroll, play video).

```
import React, { useRef } from "react";
```

```

export default function App() {
  const inputRef = useRef(null);

  function handleInputFocus() {
    inputRef.current.focus();
  }

  return (
    <div className="p-5">
      <input type="text" placeholder="type here..." ref={inputRef} />
      <button className="btn" onClick={handleInputFocus}>
        Focus Input
      </button>
    </div>
  );
}

```

Storing Mutable Values (without Re-render)

Sometimes you want to store data between renders without triggering updates.

```

import React, { useEffect, useRef, useState } from "react";

export default function App() {
  const renderCount = useRef(0);
  const [randomNum, setRandomNum] = useState(0);

  function generateRandomNumber() {
    let num = Math.floor(Math.random() * 100);
    setRandomNum(num);
  }

  useEffect(() => {
    renderCount.current += 1;
  });

  return (

```

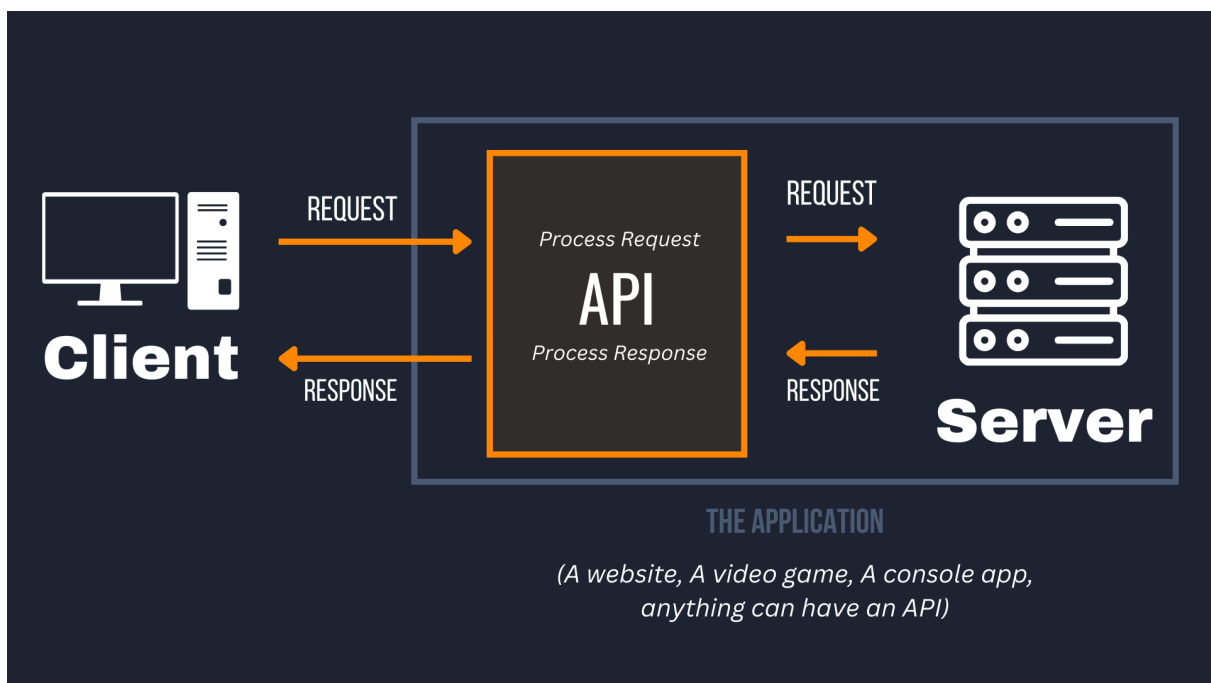
```

    <div className="p-5">
      <p className="text-2xl">Random Number is {randomNum}
    </p>
      <p className="text-xl my-4">Component rendered {rende
rCount.current} times</p>
      <button className="btn" onClick={generateRandomNumbe
r}>
        Generate Random Number
      </button>
    </div>
  );
}

```

fetch And axios In React JS

Let's try to understand client server architecture



Understanding JSON



JSON stands for JavaScript Object Notation.

It's a **lightweight data format** used to store and exchange data — like sending data between a browser and a server. Keys and string values are wrapped in double quotes. It's a **string**, not an object

```
const jsonUser = `{
  "name": "Hussain",
  "age": 21,
  "isStudent": true
}`
```

HTTP Methods

Tells a server what action to perform on a resource, such as creating, reading, updating, or deleting data.

- **GET** Get/fetch existing data.
- **POST** Add new data.
- **PUT** Modify existing data. (Replaces the entire resource)
- **PATCH** Modify existing data. (update specific fields without affecting others)
- **DELETE** Remove existing data.

fetch() Method

It is the built-in method of the browser. No installation requires to use this. It returns Promise. You needs to convert JSON into readable JavaScript Object. More code, less features in this method.

```
import React, { useEffect, useState } from "react";

export default function App() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((data) => setData(data));
  }, []);
}
```

```

if (data.length !== 0) {
  console.log(data);
}

return (
  <div className="p-5">
    <h1>Hussain Ilyas</h1>
  </div>
);
}

```

axios In React JS

It is an External library. It has Clean syntax. It Automatically parse JSON. It has Better error handling. Interceptors (request/response) also here available. Timeouts also available inside this library. It also has Upload/Download progress support.

Installation Command

```
npm install axios
```

Import **axios**

```
import axios from "axios";
```

Basic Code Example

```

import React, { useEffect, useState } from "react";
import axios from "axios";

export default function App() {
  const [data, setData] = useState([]);

  useEffect(() => {
    axios.get("https://jsonplaceholder.typicode.com/users")
      .then((res) => setData(res.data))
  });
}

```

```

        .catch((err) => console.log(err));
    }, []);

    if (data.length !== 0) {
        console.log(data);
    }

    return (
        <div className="p-5">
            <h1>Hussain Ilyas</h1>
        </div>
    );
}

```

axios In React JS

axios is a popular JavaScript library for making HTTP requests from both browsers and Node.js, simplifying communication with servers to fetch or send data.

How to install **axios**

Using CDN

```

<script src="https://cdn.jsdelivr.net/npm/axios@1.20.0/dist/axios.min.js"></script>

```

Using NPM

```

npm install axios

```

axios Configuration

These are the available config options for making requests.

```

const config = {
    url: "/users",
    baseURL: "https://jsonplaceholder.typicode.com",

```

```

method: "get",
params: {
  id: 595,
},
data: {
  name: "HSN",
  age: 21,
},
headers: {
  "Content-Type": "application/json",
  Accept: "application/json",
  Authorization: "Bearer 'Token'",
},
timeout: 5000,
};

```

Only the url is required. Requests will default to **GET** if method is not specified.



timeout is by default 0 which means no time limit.



some headers are also by default set, so you don't need to mention such headers. (you can see in browser network tab)

Different Response Types

responseType	Used For	responseType
json	API responses (default)	json
text	Plain string / HTML	text
blob	File download	blob (used for images, pdf's, videos, zip files)
arraybuffer	Low-level binary	arraybuffer (Audio processing, Video stream buffering, Custom binary protocols)
document	XML / HTML	document

Getting Data using **axios**

```
const [userData, setUserData] = useState([]);

async function handleFetchData() {
  const response = await axios({
    url: "https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users",
    method: "GET",
  });

  setUserData(response.data);
}
```

Posting Data using **axios**

```
const [userInfo, setUserInfo] = useState({ name: "", age: "" });

async function handlePostData() {
  const response = await axios({
    url: "https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users",
    method: "POST",
    headers: { "Content-Type": "application/json" },
    data: userInfo,
  });

  setUserInfo({ name: "", age: "" });
}
```

Putting (Updating) Data using **axios**

```

const [userInfo, setUserInfo] = useState({ name: "", age: "" });

async function handlePutData(userId) {
  let response = await axios({
    url: `https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users/${userId}`,
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    data: userInfo,
  });

  setUserInfo({ name: "", age: "" });
}

```

Deleting Data using **axios**

```

async function handleDeleteData(userId) {
  let response = await axios({
    url: `https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users/${userId}`,
    method: "DELETE",
  });
}

```

Full CRUD Operation Implementation

```

import React, { useEffect, useState } from "react";
import axios from "axios";

export default function App() {
  const [userData, setUserData] = useState([]);
  const [userInfo, setUserInfo] = useState({ name: "", age: "" });

  // =====

```

```

//      GET Request
// =====

async function handleFetchData() {
  const response = await axios({
    url: "https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users",
    method: "GET",
  });

  setUserData(response.data);
}

// =====
//      POST Request
// =====

async function handlePostData() {
  if (userInfo.name && userInfo.age) {
    await axios({
      url: "https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users",
      headers: { "Content-Type": "application/json" },
      method: "POST",
      data: userInfo,
    });

    setUserInfo({ name: "", age: "" });
    handleFetchData();
  } else {
    alert("Invalid data entered!");
  }
}

// =====
//      PUT Request
// =====

```

```

async function handlePutData(userId) {
  if (userInfo.name && userInfo.age) {
    let response = await axios({
      url: `https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users/${userId}`,
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      data: userInfo,
    });

    setUserInfo({ name: "", age: "" });
    handleFetchData();
  } else {
    alert("Invalid data entered!");
  }
}

// =====
//      DELETE Request
// =====

async function handleDeleteData(userId) {
  let response = await axios({
    url: `https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1/users/${userId}`,
    method: "DELETE",
  });

  handleFetchData();
}

// =====
//      Event Handlers
// =====

function handleUserInfo(e) {
  let { name, value } = e.target;
  setUserInfo((prev) => {

```

```

        return { ...prev, [name]: value };
    });
}

// =====
//      Handle Side Effects
// =====

useEffect(() => {
    handleFetchData();
}, []);

// =====
//      JSX Code
// =====

return (
    <div className="p-5">
        <div className="mb-5 flex items-center gap-2">
            <input
                type="text"
                name="name"
                placeholder="Enter your name"
                value={userInfo.name}
                onChange={handleUserInfo}
                className="border border-white/10 py-3 px-4 rounded
ed-lg outline-none"
            />
            <input
                type="number"
                name="age"
                placeholder="Enter your age"
                value={userInfo.age}
                onChange={handleUserInfo}
                className="border border-white/10 py-3 px-4 rounded
ed-lg outline-none"
            />
            {userInfo.editMode ? (

```

```

        <button className="btn" onClick={() => handlePutD
ata(userInfo.id)}>
            Edit Data
        </button>
    ) : (
        <button className="btn" onClick={handlePostData}>
            Post Data
        </button>
    )}
</div>

<div className="mt-5 flex flex-col gap-3">
    {userData.map((user, index) => (
        <div
            key={index}
            className="bg-neutral-900 p-4 rounded-lg flex j
ustify-between"
        >
            <div>
                <h3 className="text-xl capitalize text-orange
-400 font-semibold">
                    {user.name}
                </h3>
                <p className="text-gray-400 text-md font-norm
al">
                    {user.age} years old
                </p>
            </div>
            <div className="flex gap-2">
                <button
                    className="btn"
                    onClick={() => setUserInfo({ ...user, editM
ode: true })}>
                    Edit
                </button>
                <button className="btn" onClick={() => handle
DeleteData(user.id)}>

```

```

        Delete
      </button>
    </div>
  </div>
  )})
</div>
</div>
);
}

```

Shorthand Request Methods

- `axios.get(url, config)`
- `axios.post(url, data, config)`
- `axios.put(url, data, config)`
- `axios.patch(url, data, config)`
- `axios.delete(url, config)`
- `axios.head(url, config)`
- `axios.options(url, config)`

Shorthand Methods Example Code

```

import React, { useEffect, useState } from "react";
import axios from "axios";

export default function App() {
  const api_url = "https://6a92c5b925936d5660f05ee5.mockap
i.io/api/v1/users";

  const [userData, setUserData] = useState([]);
  const [userInfo, setUserInfo] = useState({
    name: "",
    age: "",
  });
}

```

```

// =====
//      GET Request
// =====

async function handleFetchData(api_url) {
  const response = await axios.get(api_url);
  setUserData(response.data);
}

// =====
//      POST Request
// =====

async function handlePostData(api_url) {
  if (userInfo.name && userInfo.age) {
    const config = { headers: { "Content-Type": "application/json" } };
    await axios.post(api_url, userInfo, config);
    setUserInfo({ name: "", age: "" });
    handleFetchData(api_url);
  } else {
    alert("Invalid data entered!");
  }
}

// =====
//      PUT Request
// =====

async function handlePutData(api_url, userId) {
  const config = { headers: { "Content-Type": "application/json" } };
  let response = await axios.put(`${api_url}/${userId}`,
  userInfo, config);
  setUserInfo({ name: "", age: "" });
  handleFetchData(api_url);
}

```

```

// =====
//      DELETE Request
// =====

async function handleDeleteData(api_url, userId) {
  let response = await axios.delete(`${api_url}/${userId}`);
  handleFetchData(api_url);
}

// =====
//      Event Handlers
// =====

function handleUserInfo(e) {
  let { name, value } = e.target;
  setUserInfo((prev) => {
    return { ...prev, [name]: value };
  });
}

// =====
//      Handle Side Effects
// =====

useEffect(() => {
  handleFetchData(api_url);
}, []);

// =====
//      JSX Code
// =====

return (
  <div className="p-5">
    <div className="mb-5 flex items-center gap-2">
      <input
        type="text"

```

```

        name="name"
        placeholder="Enter your name"
        value={userInfo.name}
        onChange={handleUserInfo}
        className="border border-white/10 py-3 px-4 rounded
ed-lg outline-none"
      />
      <input
        type="number"
        name="age"
        placeholder="Enter your age"
        value={userInfo.age}
        onChange={handleUserInfo}
        className="border border-white/10 py-3 px-4 rounded
ed-lg outline-none"
      />
      {userInfo.editMode ? (
        <button
          className="btn"
          onClick={() => handlePutData(api_url, userInfo.
id)}
        >
          Edit Data
        </button>
      ) : (
        <button className="btn" onClick={() => handlePost
Data(api_url)}>
          Post Data
        </button>
      )}
    </div>

    <div className="mt-5 flex flex-col gap-3">
      {userData.map((user, index) => (
        <div
          key={index}
          className="bg-neutral-900 p-4 rounded-lg flex j
ustify-between"

```

```
>
    <div>
      <h3 className="text-xl capitalize text-orange-400 font-semibold">
        {user.name}
      </h3>
      <p className="text-gray-400 text-md font-normal">
        {user.age} years old
      </p>
    </div>
    <div className="flex gap-2">
      <button
        className="btn"
        onClick={() => setUserInfo({...user, editMode: true })}>
        Edit
      </button>
      <button
        className="btn"
        onClick={() => handleDeleteData(api_url, user.id)}>
        Delete
      </button>
    </div>
  </div>
))}
</div>
</div>
);
}
```

Interceptors In `axios`

Injecting something in between Request and Response. Interceptors are usually created in `useEffect()` hook for better performance optimization.

Syntax

```
// =====  
//   Request Interceptors  
// =====  
  
axios.interceptors.request.use(successRequestFunc, RejectRe  
questFunc);  
  
// =====  
//   Response Interceptors  
// =====  
  
axios.interceptors.response.use(successResponseFunc, Reject  
ResponseFunc);
```

Example Code

```
// =====  
//   Request Interceptors  
// =====  
  
axios.interceptors.request.use(  
  (config) => {  
    config.headers.Authorization = `Bearer ANY_TOKEN`;  
    console.log(config.headers);  
    return config;  
  },  
  (err) => {  
    console.log(err);  
    return Promise.reject(err);  
  },  
);  
  
// =====  
//   Response Interceptors  
// =====
```

```

axios.interceptors.response.use(
  (response) => {
    response.status = 400;
    console.log(response);
    return response;
  },
  (err) => {
    console.log(err);
    return Promise.reject(err);
  },
);

```

File Upload Progress

```

// =====
//   Upload Progress
// =====

await axios.post(api_url, userInfo, {
  headers: { "Content-Type": "application/json" },
  onUploadProgress: (progress) => {
    let percentProgress = (progress.loaded / progress.total) * 100;
    console.log(`${percentProgress}%`);
  },
});

// =====
//   Download Progress
// =====

const response = await axios.get(api_url, {
  onDownloadProgress: (progress) => {
    let percentProgress = (progress.loaded / progress.total) * 100;
    console.log(`${percentProgress}%`);
  },
});

```

```
},  
});
```

Using **axios** Instance

You can mention **baseUrl**, default headers, timeout, auth token and common configuration, etc

```
import React, { useEffect, useState } from "react";  
import axios from "axios";  
  
export default function App() {  
  const [userData, setUserData] = useState([]);  
  
  // =====  
  //   Creating Axios Instance  
  // =====  
  
  const axiosInstance = axios.create({  
    baseUrl: "https://6a92c5b925936d5660f05ee5.mockapi.io/api/v1",  
    responseType: "json",  
    timeout: 5000,  
  });  
  
  // =====  
  //   Using Axios Instance  
  // =====  
  
  async function fetchData() {  
    const { data } = await axiosInstance.get("/users");  
    setUserData(data);  
  }  
  
  // =====  
  //   Handle Side Effects  
  // =====
```

```

useEffect(() => {
  fetchData();
}, []);

return (
  <div>
    <h1>Hussain</h1>
  </div>
);
}

```

React Routing - React Router DOM

React Router is a client-side routing library that allows us to create **Single Page Applications (SPA)** where Users **navigate between pages without refreshing** the page and Only the **necessary components update**, improving performance and UX.

React Router v6

- Basic Setup (`BrowserRouter` + `Routes` + `Route`)
- `Link` & `NavLink`
- Dynamic Routing
- Programmatic Navigation (`useNavigate()`)
- Redirect – Navigate component
- Nested Routing

React Router Newer Version

- Basic Setup (`createBrowserRouter` + `RouterProvider`)
- `Link` & `NavLink`
- Dynamic Routing
- Programmatic Navigation (`useNavigate()`)
- Redirect – Navigate component

- Nested Routing
- Fetch data before component loads

Installation Of React Router DOM

```
npm install react-router-dom
```

Wrapping main Component Into `<BrowserRouter>`

Inside `main.jsx` file we have to wrap the `<App />` component

```
// ***** import BrowserRouter *****
import { BrowserRouter } from "react-router-dom";

// ***** wrap the <App /> component *****
<BrowserRouter>
  <App />
</BrowserRouter>
```

Creating Different Routes

Inside `app.jsx` file we create different routes to different links

```
import React from "react";

import Header from "../components/Header";
import Home from "../components/Home";
import About from "../components/About";
import Contact from "../components/Contact";
import Help from "../components/Help";

// ***** import (Routes & Route) *****
import { Routes, Route } from "react-router-dom";

export default function App() {
  return (
    <div>
      <Header />
```

```

    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
      <Route path="/help" element={<Help />} />
    </Routes>
  </div>
);
}

```

<Link> In React Router DOM

The `Link` component is used to navigate between pages **without reloading the browser**.

```

import React from "react";

// ***** import Link *****
import { Link } from "react-router-dom";

export default function Header() {
  return (
    <header className="p-4 bg-[lightseagreen]">
      <div className="w-full flex gap-3">
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
        <Link to="/help">Help</Link>
      </div>
    </header>
  );
}

```

<NavLink> In React Router DOM

`NavLink` is similar to `Link`, but it has one **extra feature**: it knows when the current URL matches its path, so you can style the **active link**.

```

import React from "react";

// ***** import NavLink *****
import { NavLink } from "react-router-dom";

export default function Header() {
  return (
    <header className="p-4 bg-[lightseagreen]">
      <div className="w-full flex gap-3">
        <NavLink to="/" className={({ isActive }) => (isActive ? "text-orange-400" : "")}>
          Home
        </NavLink>
        <NavLink to="/about" className={({ isActive }) => (isActive ? "text-orange-400" : "")}>
          About
        </NavLink>
        <NavLink to="/contact" className={({ isActive }) => (isActive ? "text-orange-400" : "")}>
          Contact
        </NavLink>
        <NavLink to="/help" className={({ isActive }) => (isActive ? "text-orange-400" : "")}>
          Help
        </NavLink>
      </div>
    </header>
  );
}

```

Some extra features in `NavLink`

```

<NavLink
  to="/"
  className={({ features }) => {
    console.log(features);
  }}
>

```

```
Home  
</NavLink>
```

`useNavigate()` Hook

`useNavigate()` is a React Router hook used to navigate programmatically from one route to another.

```
import React from "react";  
import { useNavigate } from "react-router-dom";  
  
export default function Header() {  
  const navigate = useNavigate();  
  const isLoggedIn = true;  
  
  function handleClick() {  
    if (isLoggedIn) {  
      navigate("/about");  
    }  
  }  
  
  return (  
    <header>  
      <button className="btn" onClick={handleClick}>  
        Visit About Page  
      </button>  
    </header>  
  );  
}
```

Dynamic Routing In React JS

Making a Route Dynamic

```
import React from "react";  
import Header from "../components/Header";  
import { Routes, Route } from "react-router-dom";  
import Products from "../components/Products";
```

```
import ProductDetail from "../components/ProductDetail";

export default function App() {
  return (
    <>
      <Header />
      <Routes>
        <Route path="/products" element={<Products />} />
        { /* === Setting Up Dynamic Route === */ }
        <Route path="/products/:id" element={<ProductDetail
/>} />
      </Routes>
    </>
  );
}
```

useParams() Hook

`useParams()` is a React Router hook used to **get dynamic values from the URL**. It should be called **inside the component that is using the dynamic parameter from the URL**.

```
import React from "react";
import productData from "../data/products-data.json";
import { useParams } from "react-router-dom";

export default function ProductDetail() {
  const param = useParams();

  const productInfo = productData.find((product) => {
    return product.id === Number(param.id);
  });

  return (
    <div className="p-5">
      <h1 className="text-3xl text-orange-400">{productInf
o.name}</h1>
      <p className="mt-4">Cost {productInfo.price} - PKR</p
>
```

```

    </div>
  );
}

```

Passing or setting value (id) In URL (Param)

```

import React from "react";
import productData from "../data/products-data.json";
import { Link } from "react-router-dom";

export default function Products() {
  return (
    <div className="px-5 py-10">
      <h1 className="text-center text-3xl mb-10">Products</h1>

      { /* === Setting id in url (param) === */ }
      <div className="grid grid-cols-3 gap-6">
        {productData.map(({ id, name, price }) => (
          <Link to={` /products/${id}`} key={id}>
            <div className="bg-neutral-900 p-8 text-center rounded-xl cursor-pointer">
              <h3 className="text-2xl text-orange-400">{name}</h3>

              <p className="mt-3">{price} - PKR</p>
            </div>
          </Link>
        ))}
      </div>
    </div>
  );
}

```

Redirect Routing In React JS

First of all import `Navigate` Component from `react-router-dom`

```
import { Navigate } from "react-router-dom";
```

Implement Redirect routing

```
<Route path="/help" element={<Navigate to="/" replace={true} />} />
```

replace={true}

It tells React Router to replace the current history entry instead of adding a new one.

Without **replace**

Home → /help → /

The browser history still contains **/help**. So if the user presses the **Back** button, they may go back to **/help**.

With **replace**

Home → /

/help is replaced, so pressing **Back** won't take the user back to **/help**.

Nested Routing In React JS

Nested routing means having a route inside another route.

Define the routes

```
import React from "react";
import Header from "../components/Header";
import Home from "../components/Home";
import About from "../components/About";
import { Routes, Route } from "react-router-dom";
```

```

import HussainUser from "../components/HussainUser";
import IlyasUser from "../components/IlyasUser";

export default function App() {
  return (
    <>
      <Header />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />}>
          <Route index element={<HussainUser />} />
          <Route path="hussain" element={<HussainUser />} />
        </Route>
        <Route path="ilyas" element={<IlyasUser />} />
      </Routes>
    </>
  );
}

```

Use `<Outlet />`

This is the **most important part** of nested routing. Inside `About.jsx`

```

import React from "react";
import { Outlet } from "react-router-dom";

export default function About() {
  return (
    <div>
      <h1>About</h1>
      <Outlet />
    </div>
  );
}

```

Show Default Nested Route (`index`)

If you want one nested route to show by default when the parent route is opened, use an index route.

```
<Route index element={<HussainUser />} />
```

React Router **Newer Version**

Define the routes

create a separate file `route.jsx`

```
import Home from "../components/Home";
import About from "../components/About";
import Products from "../components/Products";
import Contact from "../components/Contact";
import Help from "../components/Help";
import Layout from "../components/Layout";
import { createBrowserRouter } from "react-router-dom";
import ProductDetail from "../components/ProductDetail";

const route = createBrowserRouter([
  {
    path: "/",
    element: <Layout />,
    children: [
      { path: "/", element: <Home /> },
      { path: "/about", element: <About /> },
      { path: "/products", element: <Products /> },
      { path: "/products/:id", element: <ProductDetail /> }
    ],
  },
  { path: "/contact", element: <Contact /> },
  { path: "/help", element: <Help /> },
],
);

export default route;
```

Providing Routes

Inside `app.jsx`

```
import React from "react";
import { RouterProvider } from "react-router-dom";
import route from "../routing/route";

export default function App() {
  return (
    <>
      <RouterProvider router={route} />
    </>
  );
}
```

Setting Outlets

Inside `Layout.jsx`

```
import React from "react";
import { Outlet } from "react-router-dom";
import Header from "../components/Header";
import Footer from "../Footer";

export default function Layout() {
  return (
    <div>
      <Header />
      <Outlet />
      <Footer />
    </div>
  );
}
```

Fetch data before component loads

Loading data before component render

```

import Home from "../components/Home";
import About from "../components/About";
import Products from "../components/Products";
import Contact from "../components/Contact";
import Help from "../components/Help";
import Layout from "../components/Layout";
import { createBrowserRouter } from "react-router-dom";
import ProductDetail from "../components/ProductDetail";

const api_url = "https://jsonplaceholder.typicode.com/users";

const route = createBrowserRouter([
  {
    path: "/",
    element: <Layout />,
    children: [
      { path: "/", element: <Home /> },
      {
        path: "/about",
        element: <About />,
        loader: () => fetch(api_url),
        HydrateFallback: () => <h2>Loading...</h2>,
      },
      { path: "/products", element: <Products /> },
      { path: "/products/:id", element: <ProductDetail /> },
    ],
    { path: "/contact", element: <Contact /> },
    { path: "/help", element: <Help /> },
  ],
],
);

export default route;

```

Getting data

```
import React from "react";
import { useLoaderData } from "react-router-dom";

export default function About() {
  const data = useLoaderData();
  console.log(data);

  return (
    <div>
      <h1>About</h1>
    </div>
  );
}
```

State Management

State Management is a way to organize, store, update and share state between different parts of the application without confusion.

Good State Management Ensures

- Data consistency
- Cleaner code
- Easy debugging
- Better performance
- Scalable architecture

Prop Drilling

Suppose we have

```
App
↓
Parent
↓
Child
```

↓
GrandChild

If `App` has some data and `GrandChild` needs it, normally we have to pass that data through `Parent` and `Child`. This is called **Prop Drilling**.

Context API In React JS

Context API is a feature in React that allows us to share data between different components without passing props manually at every level.

Main Parts of Context API

There are mainly 3 important things

- `createContext()`
- `Provider`
- `useContext()`

Creating Context

`createContext()` It creates a Context.

```
import React, { createContext } from "react";

export const MyContext = createContext();
```

Providing Context

`Provider` is used to provide/share data with other components.

```
import { MyContext } from "../context/MyContext.jsx";
import data from "../data/data.json";

<MyContext.Provider value={data}>
  <App />
</MyContext.Provider>
```

Using Context

`useContext()` is used to **get the data from Context**

```
import React, { useContext } from "react";
import { MyContext } from "../context/MyContext";

export default function Child() {
  const data = useContext(MyContext);
  console.log(data);

  return (
    <div>
      <h1>Child</h1>
    </div>
  );
}
```

Easy formula to remember

```
createContext()
  ↓
Provider
  ↓
useContext()
  ↓
Get the data
```

Production Approach (Strategy)

Suppose we have `MyContext.jsx`

```
import React, { Children, createContext } from "react";
import data from "../data/data.json";
import { useContext } from "react";

// =====
//   Create Context
// =====

const MyContext = createContext();
```

```

// =====
//   Provide Context
// =====

export function MyContextProvider({ children }) {
  return (
    <>
      <MyContext.Provider value={data}>
        {children}
      </MyContext.Provider>
    </>
  );
}

// =====
//   Using Context
// =====

export function useMyContext() {
  return useContext(MyContext);
}

```

Providing in `main.jsx`

```

import { MyContextProvider } from "../context/MyContext.js";

<MyContextProvider>
  <App />
</MyContextProvider>

```

Using context data

```

import React from "react";
import { useMyContext } from "../context/MyContext";

export default function Child() {

```

```
const data = useMyContext();
console.log(data);

return (
  <div>
    {data.map((item) => (
      <h1 key={item.id}>{item.name}</h1>
    ))}
  </div>
);
}
```



Zustand In React JS



Think of **Zustand** as a global store where you keep data that multiple components can access.

Zustand is a small and simple **state management library for React**. It makes global state management much easier.

Installation

Zustand is available as a package on NPM for use

```
npm install zustand
```

First create a store

Your store is a hook! You can put anything in it - primitives, objects, functions. The **set** function *merges* state.

```
import { create } from "zustand";

const useMyStore = create((set, get) => {
  return {
    // ***** State *****
  }
});
```

```

    count: 1,
    name: "hussain",

    // ***** Actions *****
    increment: () => {
      set((state) => ({
        count: state.count + 1,
      }));
    },
    resetCount: () => set({ count: 0 }),
    capitalizeName: () => {
      const { name } = get();
      set({ name: name.charAt(0).toUpperCase() + name.slice
(1) });
    },
  };
});

export default useMyStore;

```

Then bind your components, and that's it!

You can use the hook anywhere, without the need of providers.

```

import React, { useEffect } from "react";
import useMyStore from "../store/store";

export default function Child() {
  // ***** Choose store's state subscription carefully *****
  *
  const count = useMyStore((state) => state.count);
  const increment = useMyStore((state) => state.increment);
  const resetCount = useMyStore((state) => state.resetCoun
t);

  useEffect(() => {
    console.log("Rerender hua ha...", count);
  });
}

```

```

return (
  <div>
    <h1>Count value is {count}</h1>
    <button className="btn" onClick={increment}>
      Increment
    </button>
    <button className="btn" onClick={resetCount}>
      Reset Count
    </button>
  </div>
);
}

```

How To Persist State In Zustand

Syntax Example

```

import { create } from "zustand";
import { persist } from "zustand/middleware";

const useMyStore = create(persist((set, get) => {
  return {
    // Zustand state and actions
  }
}, {}));

```

Example Code

```

import { create } from "zustand";
import { createJSONStorage, persist } from "zustand/middleware";

const useMyStore = create(
  persist(
    (set, get) => {
      return {
        // ***** State *****
      }
    }
  )
);

```

```

    count: 1,
    name: "hussain",

    // ***** Actions *****
    increment: () => {
      set((state) => ({
        count: state.count + 1,
      }));
    },
    resetCount: () => set({ count: 0 }),
    capitalizeName: () => {
      const { name } = get();
      set({ name: name.charAt(0).toUpperCase() + name.s
lice(1) });
    },
  };
},
{
  name: "storeData",
  storage: createJSONStorage(() => localStorage),
  // storage: createJSONStorage(() => sessionStorage),
},
),
);

export default useMyStore;

```

Dev Tools In Zustand

First of all add chrome extension [Redux DevTools](#)

Then Code

```

import { create } from "zustand";
import { createJSONStorage, devtools, persist } from "zustand/middleware";

const useMyStore = create(

```

```

devtools(
  persist(
    (set, get) => {
      return {
        // ***** State *****
        count: 1,
        name: "hussain",

        // ***** Actions *****
        increment: () => {
          set((state) => ({
            count: state.count + 1,
          }));
        },
        resetCount: () => set({ count: 0 }),
        capitalizeName: () => {
          const { name } = get();
          set({ name: name.charAt(0).toUpperCase() + name
e.slice(1) });
        },
      };
    },
    {
      name: "storeData",
      storage: createJSONStorage(() => localStorage),
      // storage: createJSONStorage(() => sessionStorage)
    },
  ),
),
);

export default useMyStore;

```

As a result when you open **Redux DevTools** extension in chrome, you'll see the result 😊

useReducer() Hook

`useReducer()` is a React Hook for managing state when the update logic is more complex than a few simple `useState()` calls.

Basic Syntax

How to import

```
import React, { useReducer } from "react";
```

How to use

```
const [state, dispatch] = useReducer(reducer, initialState);
```

You provide

- `state` - current state
- `dispatch` - sends an action describing what happened
- `reducer` - function that describe how state changes
- `initialState` - starting state

Reducer Function

A `reducer` is a normal JavaScript function used to determine how the state should change.

Basic Syntax

```
function reducer(state, action) {  
  // logic to update state...  
  // here we use some conditional statements that matches  
  (action.type)...  
  return state;  
}
```

The `reducer` function receives two parameters

- `state` - represent the current state

- **action** - an object that describe what happened or what change we want to make.

What Is Dispatch

It is function, Its job is to send an action or some additional data to the **reducer** function

```
dispatch({
  type: "TYPE_OF_ACTION",
  payload: additional_data
});
```

Example Code

reducer.js file contain this code

```
export function reducer(state, action) {
  if (action.type === "INCREMENT_COUNT") {
    return { ...state, count: state.count + 1 };
  } else if (action.type === "DECREMENT_COUNT") {
    return { ...state, count: state.count - 1 };
  } else if (action.type === "RESET_COUNT") {
    return { ...state, count: 0 };
  } else {
    return state;
  }
}
```

App.jsx file contain this code

```
import React, { useReducer } from "react";
import { reducer } from "./reducer";

export default function App() {
  const initialState = { count: 0 };

  const [state, dispatch] = useReducer(reducer, initialState);
}
```

```

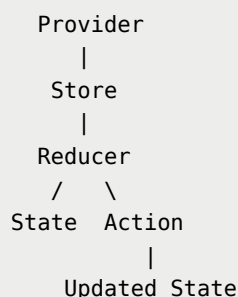
return (
  <div className="p-5 text-center">
    <h1 className="mb-2">useReducer Hook In React JS</h1>
    <h2 className="mb-4 text-xl">count value is {state.co
unt}</h2>
    <div className="flex gap-3 justify-center">
      <button onClick={() => dispatch({ type: "INCREMENT_
COUNT" })}>
        Increment
      </button>
      <button onClick={() => dispatch({ type: "DECREMENT_
COUNT" })}>
        Decrement
      </button>
      <button onClick={() => dispatch({ type: "RESET_COUN
T" })}>Reset</button>
    </div>
  </div>
);
}

```

Redux & React Redux

What Is Redux

Redux is a JavaScript library used to manage application state in a centralized **store**.



Command to install Redux

```
npm install redux
```

What Is **React Redux**

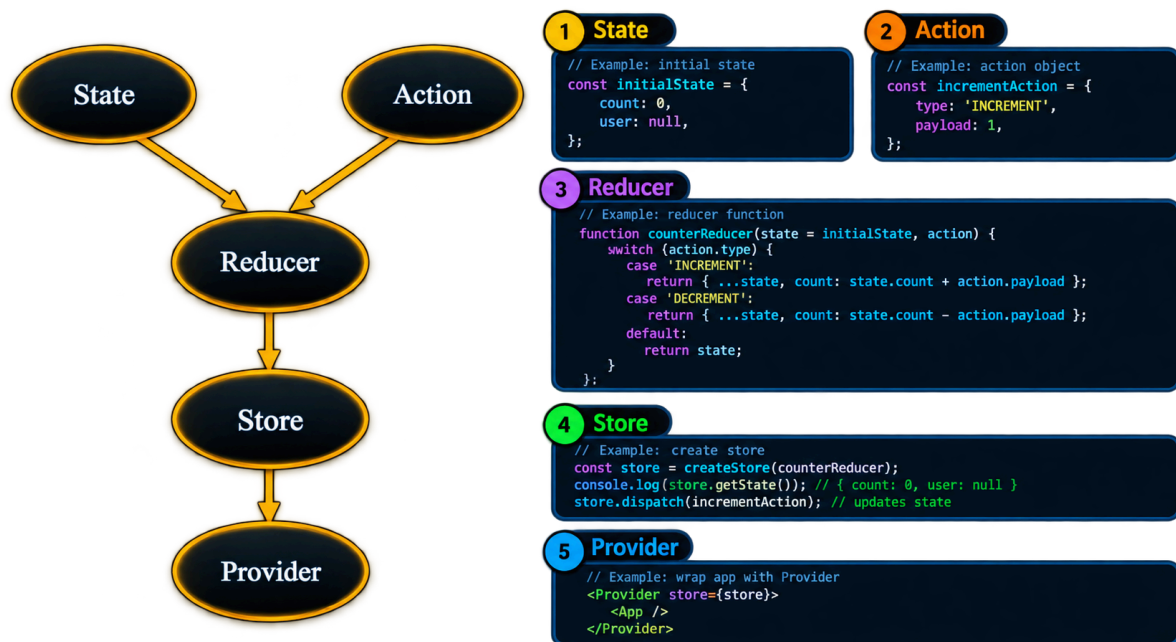
React-Redux is a library that connects **Redux with React**.

Command to install React Redux

```
npm install react-redux
```

The main things React-Redux provides are

- **Provider**
- **useSelector()**
- **useDispatch()**



Steps To Manage State With Redux And React Redux

Create Store

Create `store.js` file

```
import { createStore } from "redux";
import { productReducer } from "../reducers/productReducer";

const store = createStore(
  productReducer
);

export default store;
```

Provide Store

Inside `main.jsx` file

```
import { Provider } from "react-redux";
import store from "../redux/store.js";

<Provider store={store}>
  <App />
</Provider>
```

Create Reducer

Create `productReducer.js` file. create a reducer function in this file and provide this reducer function in the store that you create using `createStore` from redux

```
const initialState = {
  products: [
    { id: 1, name: "computer", price: 46_000 },
    { id: 2, name: "mobile", price: 18_000 },
    { id: 3, name: "watch", price: 7_000 },
  ],
};

export function productReducer(state = initialState, action) {
  if (action.type === "ADD_PRODUCT") {
    return {
```

```

        products: [...state.products, action.payload],
    };
} else if (action.type === "REMOVE_PRODUCT") {
    return {
        products: state.products.filter((prod) => {
            return prod.name !== action.payload;
        }),
    };
} else if (action.type === "UPDATE_PRODUCT") {
    return {
        products: state.products.map((prod) => {
            if (prod.name === "watch") {
                return { ...prod, ...action.payload };
            } else {
                return prod;
            }
        }),
    };
} else {
    return state;
}
}

```

Create Action

Create `productAction.js` file. and write each action that you want to perform by the reducer

```

export function addProduct(newProduct) {
    return {
        type: "ADD_PRODUCT",
        payload: newProduct,
    };
}

export function removeProduct(productName) {
    return {
        type: "REMOVE_PRODUCT",
    };
}

```

```

        payload: productName,
    };
}

export function updateProduct(newDetail) {
    return {
        type: "UPDATE_PRODUCT",
        payload: newDetail,
    };
}

```

Using State And Dispatching Actions

In order to use state we use `useSelector()` hook and to dispatch action we use `useDispatch()` hook. Considered the following example

```

import React from "react";
import { useSelector, useDispatch } from "react-redux";
import {
    addProduct,
    removeProduct,
    updateProduct,
} from "../redux/actions/productAction";

export default function App() {
    // Take the subscription carefully to reduce unnecessary
    // rerender
    const products = useSelector((state) => state.products);
    const dispatch = useDispatch();

    function handleAddProduct() {
        const newProduct = { id: 4, name: "laptop", price: 32_000 };
        dispatch(addProduct(newProduct));
    }

    function handleDeleteProduct() {
        dispatch(removeProduct("computer"));
    }
}

```

```

    }

    function handleUpdateProduct() {
        const newDetail = { name: "smartwatch", price: 15_000 };
        dispatch(updateProduct(newDetail));
    }

    return (
        <div>
            <h1>Total products are {products.length}</h1>

            <button onClick={handleAddProduct} className="btn">
                Add Product
            </button>

            <button onClick={handleDeleteProduct} className="btn">
                Delete Product
            </button>

            <button onClick={handleUpdateProduct} className="btn">
                Update Product
            </button>
        </div>
    );
}

```

Combine Multiple Reducers Into One Root Reducer

Redux provides a function called

```
combineReducers();
```

It combines multiple reducers into **one root reducer**.

Suppose we have reducer

```
// counterReducer.js

const initialState = {
  count: 0,
};

const counterReducer = (state = initialState, action) => {
  switch (action.type) {
    case "INCREMENT":
      return {
        ...state,
        count: state.count + 1,
      };

    default:
      return state;
  }
};

export default counterReducer;
```

Another reducer

```
// userReducer.js

const initialState = {
  user: null,
};

const userReducer = (state = initialState, action) => {
  switch (action.type) {
    case "SET_USER":
      return {
        ...state,
        user: action.payload,
      };

    default:
```

```

        return state;
    }
};

export default userReducer;

```

Combine multiple reducer

```

import { combineReducers } from "redux";

import counterReducer from "./counterReducer";
import userReducer from "./userReducer";

const rootReducer = combineReducers({
  counter: counterReducer,
  user: userReducer,
});

export default rootReducer;

```

Now Our Redux state will look like

```

{
  counter: {
    count: 0
  },
  user: {
    user: null
  }
}

```

Connect **DevTools** with Redux

To connect redux with **DevTools** extension you have to specify the second parameter of `createStore(reducer, DevToolCMD)`

```
import { createStore } from "redux";
import rootReducer from "../rootReducer";

const store = createStore(
  rootReducer,
  window.__REDUX_DEVTOOLS_EXTENSION__ &&
    window.__REDUX_DEVTOOLS_EXTENSION__()
);

export default store;
```

Setting Up Some Middleware + **DevTools** With Redux

```
import { createStore, compose, applyMiddleware } from "redux";
import { productReducer } from "../reducers/productReducer";

const store = createStore(
  productReducer,
  compose(
    applyMiddleware(),
    window.__REDUX_DEVTOOLS_EXTENSION__ &&
      window.__REDUX_DEVTOOLS_EXTENSION__()
  ),
);

export default store;
```

Redux State Persistence

Redux State Persistence means saving the Redux store data so that it is not lost when the page is refreshed or the browser is closed/reopened.

Install Redux Persist

```
npm install redux-persist
```

In `store.js` file we have

```
import { createStore } from "redux";
import rootReducer from "../reducers/rootReducer";
import local from 'redux-persist/lib/storage' // defaults to
localStorage
import { persistStore, persistReducer } from "redux-persist";

const persistConfig = {
  key: "root",
  storage: local,
  blacklist: ["products"],
};

const persistedReducer = persistReducer(persistConfig, rootReducer);

export const store = createStore(persistedReducer);
export const persistor = persistStore(store);
```

In `main.jsx` file we have

```
import { Provider } from "react-redux";
import { store, persistor } from "../redux/store.js";
import { PersistGate } from "redux-persist/integration/react";

<Provider store={store}>
  <PersistGate persistor={persistor}>
    <App />
  </PersistGate>
</Provider>
```

What Is Redux **Thunk**

Redux itself expects **actions** to be plain **synchronous objects**. Redux **Thunk** is middleware that allows us to dispatch functions, so those functions can perform **asynchronous operations** and then dispatch normal Redux actions.

Why do we need Redux **Thunk**

Normally Redux actions are synchronous.

For example

```
dispatch({
  type: "SET_USER",
  payload: user
});
```

Its flow becomes

```
dispatch(action)
  ↓
  reducer
  ↓
updated state
```

Instead of dispatching an object, **Thunk** allows you to dispatch a function

```
dispatch((dispatch) => {
  // async work
});
```

So the flow becomes

```
Component
  ↓
dispatch(thunk function)
  ↓
Redux Thunk middleware
  ↓
API call
  ↓
dispatch(normal action)
  ↓
Reducer
```

↓
Store

Install Redux **Thunk**

```
npm install redux-thunk
```

Setup **Thunk** In Store

```
import { createStore, applyMiddleware } from "redux";
import { thunk } from "redux-thunk";
import rootReducer from "../reducers/rootReducer";

const store = createStore(
  rootReducer,
  applyMiddleware(thunk)
);

export default store;
```

Create **Thunk** Action

```
export const getUsers = () => {
  return async (dispatch) => {

    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );

    const data = await response.json();

    dispatch({
      type: "SET_USERS",
      payload: data
    });
  };
};
```

```
};  
};
```

Redux Toolkit In React JS

Redux Toolkit (RTK) is the official recommended way to write Redux logic.

Redux Toolkit	React Redux
It is a state management library over redux. ☐ <code>configureStore</code> ☐ <code>createSlice</code> ☐ <code>createAsyncThunk</code> ☐ Built-in <code>Immer</code> ☐ <code>DevTools</code> + <code>Thunk</code> by default	It is used to connect RTK with React. ☐ <code>Provider</code> ☐ <code>useSelector()</code> ☐ <code>useDispatch()</code>



Reducer + Actions + State = **Slice**

Reducer looks mutating, but it is immutable `Immer`

Normally in Redux, we should NOT mutate state, Suppose we have

```
const initialState = {  
  count: 0  
};
```

In traditional Redux, you would write

```
return {  
  ...state,  
  count: state.count + 1  
};
```

But Redux Toolkit lets you write this

```
const counterSlice = createSlice({  
  name: "counter",
```

```

    initialState: {
      count: 0
    },
    reducers: {
      increment: (state) => {
        state.count += 1;
      }
    }
  }
});

```

This looks like mutation, But we aren't actually mutating the original Redux state. That's where **Immer** comes in.

What is **Immer**

Redux Toolkit uses a **library** called **Immer** internally. **Immer** allows you to write code that looks like you're modifying the state directly, But **Immer** actually creates a new immutable state behind the scenes

Redux Toolkit Installation

Add the Redux Toolkit and React Redux packages to your project

```
npm install @reduxjs/toolkit react-redux
```

Create a Redux Store

Import the **configureStore** API from Redux Toolkit. We'll start by creating an empty Redux store, and exporting it

```

import { configureStore } from "@reduxjs/toolkit";

const store = configureStore({
  reducer: {},
});

export default store;

```

Provide the Redux Store to React

Once the store is created, we can make it available to our React components by putting a React-Redux `<Provider>` around our application in `src/index.js`. Import the Redux store we just created, put a `<Provider>` around your `<App>`, and pass the store as a prop

```
import { Provider } from "react-redux";
import store from "../store/store.js";

<Provider store={store}>
  <App />
</Provider>
```

Create a Redux State Slice

Creating a slice requires a string name to identify the slice, an initial state value, and one or more reducer functions to define how the state can be updated.

```
import { createSlice } from "@reduxjs/toolkit";

const state = {
  count: 0,
};

const counterSlice = createSlice({
  name: "counter",
  initialState: state,
  reducers: {
    increment: (state, action) => {
      state.count += 1;
    },
    decrement: (state, action) => {
      state.count -= 1;
    },
    changeByValue: (state, action) => {
      state.count = action.payload;
    },
  },
});
```

```
});

export const { increment, decrement, changeByValue } = counterSlice.actions;
export default counterSlice.reducer;
```

Add Slice Reducers to the Store

We need to import the reducer function from the counter slice and add it to our store

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../slices/counterSlice";

const store = configureStore({
  reducer: {
    counter: counterReducer,
  },
});

export default store;
```

Use Redux State and Actions in React Components

Now we can use the React-Redux hooks to let React components interact with the Redux store. We can read data from the store with `useSelector`, and dispatch actions using `useDispatch`.

```
import React from "react";
import { useDispatch, useSelector } from "react-redux";
import {
  changeByValue,
  decrement,
  increment,
} from "../store/slices/counterSlice";

export default function App() {
  const countState = useSelector((state) => state.counter.c
```

```

ount);
  const dispatch = useDispatch();

  return (
    <div className="text-center py-10">
      <h1 className="text-5xl font-bold">{countState}</h1>
      <button className="btn m-4" onClick={() => dispatch(increment())}>
        Increment
      </button>
      <button className="btn m-4" onClick={() => dispatch(decrement())}>
        decrement
      </button>
      <button className="btn m-4" onClick={() => dispatch(changeByValue(10))}>
        Increment by 10
      </button>
    </div>
  );
}

```

Asynchronous Working In Redux Toolkit

The function below is called a **thunk** and allows us to perform **async** logic.

```

import { createAsyncThunk, createSlice } from "@reduxjs/toolkit";
import axios from "axios";

// =====
//      Async Thunk
// =====

const API_URL = "https://jsonplaceholder.typicode.com/users";

export const fetchUser = createAsyncThunk(

```

```

    "user/fetchUser",
    async (payload, thunkAPI) => {
      const { rejectWithValue } = thunkAPI;
      try {
        const response = await axios.get(API_URL);
        return response.data;
      } catch (err) {
        return rejectWithValue("Something went wrong");
      }
    },
  );

  // =====
  //      userSlice
  // =====

  const state = {
    user: [],
    loading: false,
    error: null,
  };

  const userSlice = createSlice({
    name: "user",
    initialState: state,
    reducers: {},
    extraReducers: (addBuilder) => {
      addBuilder.addCase(fetchUser.pending, (state, action) => {
        state.loading = true;
        state.error = null;
      });
      addBuilder.addCase(fetchUser.fulfilled, (state, action) => {
        state.loading = false;
        state.user = action.payload;
      });
      addBuilder.addCase(fetchUser.rejected, (state, action)

```

```
=> {  
  state.loading = false;  
  // state.error = action.error.message;  
  state.error = action.payload;  
});  
},  
});  
  
export default userSlice.reducer;
```

Material UI / MUI In React JS

Material UI is an open-source React component library that implements Google's Material Design. It's comprehensive and can be used in production out of the box.

Installation

Run the following commands to add Material UI to your project

```
npm install @mui/material @emotion/react @emotion/styled
```

Material UI Icons Installation

```
npm install @mui/icons-material
```

Read Documentation

Visit the following URL to read documentation

```
https://mui.com/material-ui/getting-started/
```

HOC v/s Custom Hooks

In React, HOC (Higher-Order Component) and Custom Hook can both be used to reuse logic, but they do it in different ways.

HOC — Higher-Order Component

A HOC is a function that starts with `with` and its name written in `camelCase` and it takes a component and returns a new component.

Example Code

Considered you have a `UserCard.jsx` component

```
import React from "react";

export default function UserCard({name, age}) {
  return (
    <div>
      <h1>{name}</h1>
      <h4>{age} years old</h4>
      <p>
        Lorem ipsum dolor sit amet, consectetur adipiscing
        elit. Expedita,
        magnam.
      </p>
    </div>
  );
}
```

Now create a `withCardLook.jsx` Higher Order Component That return another component

```
import React from "react";

export default function withCardLook(Comp) {
  return function (props) {
    return (
      <div className="p-10 bg-gray-800 rounded-2xl text-center">
        <Comp {...props} />
      </div>
    );
  };
}
```

```
};  
}
```

Now In `App.jsx` call the `withCardLook.jsx` HOC and pass `UserCard.jsx` Component as parameter In It.

```
import React from "react";  
import withCardLook from "../HOC/withCardLook";  
import UserCard from "../components/UserCard";  
  
export default function App() {  
  const StyledUserCard = withCardLook(UserCard);  
  
  return (  
    <div className="p-5">  
      <StyledUserCard name="Hussain" age={21} />  
    </div>  
  );  
}
```

Custom Hooks In React JS

A Custom Hook is a function that starts with `use` and its name written in `camelCase` that allows you to reuse React logic/state.

Example Code

Create a `useCounter.js` custom hook and write your counter logic.

```
import { useState } from "react";  
  
export function useCounter(initialValue) {  
  const [count, setCount] = useState(initialValue);  
  
  function handleIncrement() {  
    setCount(count + 1);  
  }  
}
```

```

function handleDecrement() {
  setCount(count - 1);
}

function handleSetbyValue(val) {
  setCount(val);
}

return {
  value: count,
  increment: handleIncrement,
  decrement: handleDecrement,
  setByValue: handleSetbyValue,
};
}

```

Now use this custom hook where you want to implement counter logics.

```

import React from "react";
import { useCounter } from "../custom_hooks/useCounter";

export default function App() {
  const { value, increment, decrement, setByValue } = useCounter(10);

  return (
    <div className="p-5">
      <h1>{value}</h1>
      <button className="btn" onClick={increment}>
        Increment
      </button>
      <button className="btn" onClick={decrement}>
        Decrement
      </button>
      <button className="btn" onClick={() => setByValue(100)}>
        Set value 100
      </button>
    </div>
  );
}

```

```
    </div>
  );
}
```

React Life Cycle Methods

In React, lifecycle methods describe what happens to a component during its life



Mount → Update → Unmount

These methods are mainly associated with class components. In modern React, Hooks like `useEffect()` are used instead.

Mounting — Component is created

When a component is added to the DOM

```
import React, { useEffect } from "react";

export default function App() {
  // This is mounting phase []
  useEffect(() => {
    console.log("Side effect task rendered!");
  }, []);

  return (
    <div className="p-5">
      <h1>Hussain</h1>
    </div>
  );
}
```

Updating — Component changes

When state or props change

```

import React, { useEffect, useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);
  const [inputVal, setInputVal] = useState("");

  // This is Updating phase [state/props]
  useEffect(() => {
    console.log("Side effect task rendered!");
  }, [count, inputVal]);

  return (
    <div className="p-5">
      <h1>{count}</h1>

      <p>{inputVal}</p>

      <input
        type="text"
        className="border border-white/10"
        value={inputVal}
        onChange={(e) => setInputVal(e.target.value)}
      />

      <button className="btn" onClick={() => setCount(count
+ 1)}>
        Increment
      </button>
    </div>
  );
}

```

Unmounting — Component is removed

When the component is removed from the DOM

Example Code

Considered we have following `User.jsx` component

```
import React, { useEffect, useState } from "react";

export default function User() {
  const [name, setName] = useState(localStorage.getItem("userName") || "");

  function handleUserName(e) {
    setName(e.target.value);
    localStorage.setItem("userName", e.target.value);
  }

  // cleanup task should be perform In Unmounting Phase
  useEffect(() => {
    return () => {
      setName("");
      localStorage.clear();
    };
  }, []);

  return (
    <div className="p-10 text-center bg-gray-800 rounded-2xl">
      <h2>{name}</h2>
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
      <input
        type="text"
        className="border border-white/10 mt-2 px-4 py-2 rounded-xl"
        placeholder="Enter your name"
        value={name}
        onChange={handleUserName}
      />
    </div>
  );
}
```

In `App.jsx` Component we have

```
import React, { useState } from "react";
import User from "../components/User";

export default function App() {
  const [showUser, setShowUser] = useState(false);

  return (
    <div className="p-5">
      {showUser && <User />}
      <button className="btn" onClick={() => setShowUser(!showUser)}>
        {showUser ? "Hide User" : "Show User"}
      </button>
    </div>
  );
}
```

`useLayoutEffect()` Hook

`useLayoutEffect()` is basically a special version of `useEffect()` that runs at a different time.

First understand the timing

With normal `useEffect()`

```
Render
↓
DOM updated
↓
Browser PAINTS screen 👁️
↓
useEffect()
```

With `useLayoutEffect()`

```
Render
↓
```

```
DOM updated
↓
useLayoutEffect()
↓
Browser PAINTS screen 🙄
```

Common examples

- Tooltip positioning
- Dropdown positioning
- Element ki width/height measure karna
- Scroll position set karna
- DOM measurement ke basis par layout adjust karna
- Visual flickering prevent karna



API call, data fetching, timer, event listener → `useEffect()`



DOM measurement + immediate visual adjustment → `useLayoutEffect()`

A practical example

```
import { useEffect, useLayoutEffect, useRef, useState } from "react";

export default function App() {
  const [showTooltip, setShowTooltip] = useState(false);
  const [top, setTop] = useState(0);
  const tooltipRef = useRef(null);

  // *** It works smoothly ***
  useLayoutEffect(() => {
    if (showTooltip) {
      const height = tooltipRef.current.getBoundingClientRect().height;
      setTop(100 - height);
    }
  });
}
```

```

    }
    }, [showTooltip]);

    // *** Here is glitch ***
    // useEffect(() => {
    //   if (showTooltip) {
    //     const height = tooltipRef.current.getBoundingClien
    tRect().height;
    //     setTop(100 - height);
    //   }
    // }, [showTooltip]);

    return (
      <div className="p-5">
        <button className="btn" onClick={() => setShowTooltip
(!showTooltip)}>
          {showTooltip ? "Hide Tooltip" : "Show Tooltip"}
        </button>

        {showTooltip && (
          <div
            ref={tooltipRef}
            className="bg-gray-800 px-4 py-3 rounded-xl"
            style={{
              position: "absolute",
              top: `${top}px`,
            }}
          >
            <p>This is tooltip!</p>
          </div>
        )}
      </div>
    );
  }
}

```

React Memo

`React.memo()` is a **higher-order component (HOC)** used to optimize the performance of a React application. Its main purpose is Prevent a component from re-rendering when its props have not changed.

First Understand The Problem

Suppose we have a child component

```
import { useEffect } from "react";

export default function Child() {
  useEffect(() => {
    console.log("Child component rendering...");
  });

  return (
    <div>
      <h1>This is child component</h1>
    </div>
  );
}
```

And we have a parent component, and we called a child component in it.

```
import React, { useState } from "react";
import Child from "../components/Child";

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <Child />
      <h1>{count}</h1>
      <button className="btn" onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

```
);  
}
```

When you click the button

```
count changes  
  ↓  
App re-renders  
  ↓  
Child also re-renders
```

Even though the `Child` component's `name` prop is still `"Hussain"`. This can become a performance problem if `Child` is expensive to render.

Using `React.memo()`

We can wrap the child component with `React.memo()`

```
import { memo, useEffect } from "react";  
  
function Child({ name }) {  
  useEffect(() => {  
    console.log("Child component rendering...");  
  });  
  
  return (  
    <div>  
      <h1>This is child component</h1>  
      <p>Hye! {name}</p>  
    </div>  
  );  
}  
  
// Using memo()  
const EnhancedComponent = memo(Child);  
export default EnhancedComponent;
```

Custom Comparison In `memo()`

By default `memo()` uses **shallow** prop comparison. You can provide your own comparison function

```
import { memo, useEffect } from "react";

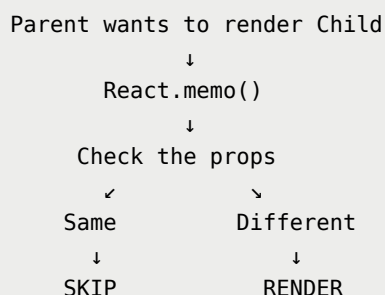
function Child({ name }) {
  useEffect(() => {
    console.log("Child component rendering...");
  });

  return (
    <div>
      <h1>This is child component</h1>
      <p>Hye! {name}</p>
    </div>
  );
}

// Using memo()
const EnhancedComponent = memo(Child, (prevProps, nextProps) => {
  return prevProps.name === nextProps.name;
});

export default EnhancedComponent;
```

The second function decides whether the component should skip rendering, If it returns `true` React considers the props equal and can skip the render, If it returns `false` React renders the component



Reference Stability

Reference stability means that a value keeps the same object/function reference between renders.

Primitive values

Primitive values are compared by value

```
const name = "Hussain";
```

If the next render also has

```
const name = "Hussain";
```

React sees them as equal

```
"Hussain" === "Hussain"; // → true
```

Objects are different

Consider

```
const user = { name: "Hussain" };
```

On every render, if you create it again

```
const user = { name: "Hussain" };
```

it's a new object

```
{ name: "Hussain" } === { name: "Hussain" }; // false
```

Even though the contents are identical, So the reference is not stable.

Making the reference stable

Use `useState()`, `useMemo()`, `useCallback()`

Considered a parent component

```
import React, { useState } from "react";
import Child from "../components/Child";

export default function App() {
  const [count, setCount] = useState(0);
  const [user, setUser] = useState({ name: "Hussain" });

  return (
    <div>
      {/* Stable object reference */}
      <Child user={user} />

      <h1>{count}</h1>

      <button className="btn" onClick={() => setCount(count
+ 1)}>
        Increment
      </button>
    </div>
  );
}
```

And a child component

```
import { memo, useEffect } from "react";

function Child({ user }) {
  useEffect(() => {
    console.log("Child component rendering...");
  });

  return (
    <div>
      <h1>This is child component</h1>
      <p>Hye! {user.name}</p>
    </div>
  );
}
```

```

    );
  }

  // memo()
  export default memo(Child, (prevProps, nextProps) => {
    return prevProps.user === nextProps.user;
  });

```

useMemo() Hook

`useMemo()` is a React Hook used to **remember or memoize the result of a calculation** so React doesn't have to calculate it again on every render.

Basic Syntax

```

const result = useMemo(() => {
  return expensiveCalculation(data);
}, [data]);

```

React will recalculate the value only when data changes.

Example Code

```

import React, { useMemo, useState } from "react";

export default function CountComp() {
  const [count, setCount] = useState(0);

  function expensiveCalc() {
    console.log("performing expensive calculation...");
    let a = 10, b = 20, c = 30;
    let result = a + b + c;
    return result;
  }

  // useMemo(func, [dependency])
  const expensiveCalcResult = useMemo(() => {

```

```

    const res = expensiveCalc();
    return res;
  }, []);

  return (
    <div>
      <p>Result = {expensiveCalcResult}</p>
      <h1>{count}</h1>
      <button className="btn" onClick={() => setCount(count
+ 1)}>
        Increment
      </button>
    </div>
  );
}

```

useCallback() Hook

`useCallback()` is a React Hook used to **remember or memoize a function** so that the same function reference can be reused between renders



On every re-render = functions are recreated



Even if the logic same - function reference changes

Basic Syntax

```

const functionName = useCallback(() => {
  // code
}, [dependencies]);

```

React creates a new function only when one of the dependencies changes.

Example Code

Considered we have a child component

```
import { memo, useEffect } from "react";

function CountComp({ handleClick }) {
  useEffect(() => {
    console.log("count component rendered...");
    handleClick();
  });

  return (
    <div>
      <h1>CountComp</h1>
    </div>
  );
}

export default memo(CountComp);
```

And now in parent component we call our child component.

```
import React, { useCallback, useState } from "react";
import CountComp from "../components/CountComp";

export default function App() {
  const [count, setCount] = useState(0);

  // normal function - recreated on every render
  function handleClick() {
    console.log("click function performed...");
  }

  // useCallback() Hook - memoized the function reference
  const memoizedHandleClick = useCallback(() => {
    return handleClick();
  }, []);

  return (
```

```

    <div className="p-5">
      <CountComp handleClick={memoizedHandleClick} />
      <h1>Hussain</h1>
      <button className="btn" onClick={() => setCount(count
+ 1)}>
        Increment {count}
      </button>
    </div>
  );
}

```

forwardRef in React

In legacy code bases (before react version 19) `forwardRef` is used when you want to **pass a `ref` from a parent component to a DOM element inside a child component**.

Normally, `ref` doesn't automatically pass through a custom component.

Code Example

considered you have a parent component

```

import React, { useRef } from "react";
import CompA from "../components/CompA";

export default function App() {
  const elementRef = useRef(null);

  function handleClick() {
    elementRef.current.focus();
    elementRef.current.value = "Hussain";
  }

  return (
    <div className="p-5">
      <CompA ref={elementRef} />
      <button className="btn" onClick={handleClick}>
        Click
      </button>
    </div>
  );
}

```

```

        </button>
      </div>
    );
  }

```

And a child component with `forwardRef`

```

import React, { forwardRef } from "react";

function CompA(props, ref) {
  return (
    <div className="my-5">
      <input className="border border-white/10" type="text"
ref={ref} />
    </div>
  );
}

export default forwardRef(CompA);

```

Code Splitting In React JS

Normally React apps bundle all JavaScript into one big file

As the app grows, bundle size become huge → slow loading time.

Code Splitting means → Load only the code that is needed right now, not everything at once

Benefits

- Fast initial load
- Better performance
- Less JS download
- Better user experience

Types Of Code Splitting

There are following types of code splitting

- Component-based splitting
- Route-based splitting
- Library splitting

Lazy Loading

React provides `lazy()` to load a component only when it is needed

```
import { lazy } from "react";

const About = lazy(() => import("./About"));
```

Instead of

```
import About from "./About";
```

The `lazy()` version creates a separate chunk for `About`

Suspense

`Suspense` displays a fallback while the lazy component is being downloaded

```
import { lazy, Suspense } from "react";

const About = lazy(() => import("./About"));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <About />
    </Suspense>
  );
}
```

Component Based Splitting

Here, you split individual components into separate chunks

For example, suppose you have

```
App
├─ Navbar
├─ Home
├─ About
└─ HeavyChart
```

Maybe `HeavyChart` is large and isn't immediately needed, You can split it

```
import { lazy, Suspense } from "react";
import Navbar from "./Navbar";
import Home from "./Home";

const HeavyChart = lazy(() => import("./HeavyChart"));

function App() {
  return (
    <>
      <Navbar />
      <Home />

      <Suspense fallback={<p>Loading chart...</p>}>
        <HeavyChart />
      </Suspense>
    </>
  );
}
```

Route Based Splitting

This is one of the most common and useful forms of code splitting.

Imagine your application has

```
Home
About
Dashboard
Settings
Profile
```

There is no reason to download all five pages when the user initially visits Home. Instead

```
Initial load
  ↓
Home.js
  ↓
User clicks Dashboard
  ↓
Dashboard.js downloaded
```

Example with React Router

```
import { lazy, Suspense } from "react";
import { Routes, Route } from "react-router-dom";

const Home = lazy(() => import("./pages/Home"));
const About = lazy(() => import("./pages/About"));
const Dashboard = lazy(() => import("./pages/Dashboard"));

function App() {
  return (
    <Suspense fallback={<p>Loading page...</p>}>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/dashboard" element={<Dashboard />} />
      </Routes>
    </Suspense>
  );
}
```

Now each page can become its own chunk

Library Based Splitting

Sometimes a third-party library is very large.

Example Code

```
const loadChart = async () => {
  const chart = await import("some-large-chart-library");

  // use chart here
};
```

Virtualization In React JS

Virtualization in React is a technique used to efficiently render large lists or tables

Basic Architecture

considered you have parent component

```
import React from "react";
import ListRendering from "../components/ListRendering";

export default function App() {
  // Array of 1000 items
  const arr = Array.from({ length: 1000 }, (_, index) => index + 1);

  return (
    <div className="p-5">
      <h1>Hussain</h1>
      <ListRendering list={arr} height={450} itemHeight={30} />
    </div>
  );
}
```

Then child component

```
import React, { useState } from "react";

function ListRendering({ list, height, itemHeight }) {
```

```

const noOfVisibleItem = Math.floor(height / itemHeight);
const [indices, setIndices] = useState([0, noOfVisibleItem]);
const visibleList = list.slice(indices[0], indices[1]);

function handleListScroll(e) {
  const { scrollTop } = e.target;
  let newStartIndex = Math.floor(scrollTop / itemHeight);
  let newEndIndex = newStartIndex + noOfVisibleItem;
  setIndices([newStartIndex, newEndIndex]);
}

return (
  <div
    style={{ height: height, overflowY: "auto", background: "black" }}
    onScroll={handleListScroll}
  >
    <div style={{ height: list.length * itemHeight }}>
      <div style={{ transform: `translateY(${indices[0] * itemHeight}px)` }}>
        {visibleList.map((item, index) => (
          <p
            style={{ height: itemHeight }}
            className="border-b border-white/10 mt-1"
            key={index}
          >
            Item - {item}
          </p>
        ))}
      </div>
    </div>
  </div>
);
}

export default ListRendering;

```

Virtualization With Library

There are some libraries for virtualization in React

- React Virtuoso (Recommended)
- React Window